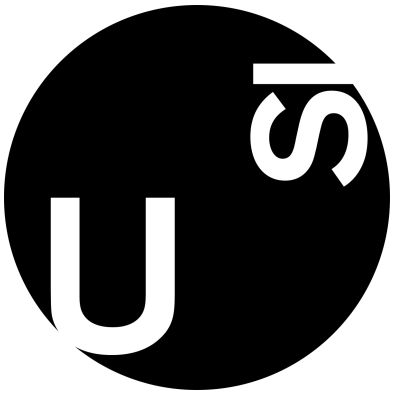


Parallel implementation of mesh-free operators for 2D and 3D PDEs on a sphere for atmospheric dynamics

Master Thesis

Devulapalli Lakshmi Aparna

January 18, 2024

Advisor: Dr. William Sawyer

Co-Advisors: Prof. Dr. Olaf Schenk, Prof. Dr. Harald Koestler

Faculty of Informatics, USI Lugano

Abstract

This thesis project explores a mesh-free method for the approximations of numerical operators for 2D and 3D partial differential equations used in atmospheric dynamics. Compactly supported radial basis functions were chosen as the type of mesh-free method for spacial discretization.

The primary objective of this project is to formulate a completely parallel algorithm, specifically developed keeping in mind the computational capabilities of a super computing node.

The programming language used for this project is Python and external libraries are integrated into the code for computationally complex tasks. Atlas, a software developed by ECMWF, was used for the domain decomposition of the points on the sphere. It provides additional functionalities to obtain the neighborhood of points using a specific radius as well as halo exchange between the processes. Other powerful libraries for performing dense linear algebraic computations were also used, like BLAS. The secondary objective of this project would be to test the capabilities of this industrial software for high-resolution grid sizes.

The results are tested using standard test cases provided by David J. Williamson et al. for Numerical Approximations of shallow water equations on a spherical geometry.[30] In conclusion, the thesis project aims to provide a package of powerful tools capable of solving 2D and 3D PDE related to atmospheric dynamics, such as shallow water equations, hydrostatic primitive equations, and non-hydrostatic fully-compressible Euler equations.

Keywords: Shallow Water Equations, Wendland function, compactly supported Radial Basis Functions, Atmosphere dynamics, Parallel Computing

Acknowledgements

I would like to extend my heartfelt gratitude towards Dr. William Sawyer. It was a great privilege to get accepted as his student as part of my thesis project. He was patient, kind and generous with imparting his knowledge throughout the course of the project. It was an extremely collaborative project and I am grateful for the experience I had working with him. His vision for the project was the guiding light for me.

A big thank you to Prof Olaf Schenk, for accepting the role as my advisor and allowing me to pursue my interest in weather prediction modelling and climate research. I am forever indebted to him for the efforts he put in finding me an external advisor for the topic of my interest. He has constantly supported my studies at USI as part of the double degree program with FAU and his reputable presence in my thesis project, has made processes tremendously smoother.

I am grateful to Prof. Harald Koestler for accepting the position of co-advisor from FAU, my home university. I am blessed to receive his guidance and support for the completion of my master's degree.

I would like to thank Willem Deconinck, scientist at ECMWF, for his precious time and expertise for developing the necessary functionalities required for this project from Atlas library. The features developed by him, formed the backbone of this project.

I could not have completed my master thesis project without the constant emotional support provided by my parents and my partner. Their confidence in me propels me to reach for newer heights.

Contents

Acknowledgements **iii**

Contents **v**

List of Figures **1**

1 Introduction **5**
1.1 Numerical Weather and Climate Prediction 6
1.1.1 History 6
1.1.2 Classification 6
1.2 The Contributing Quad 7
1.3 Mesh-free Methods 9

2 Radial Basis Functions **11**
2.1 Classification 12
2.1.1 Infinitely Smooth RBF 12
2.1.2 Polyharmonic Splines 13
2.1.3 Thin plate spline 13
2.1.4 Compactly supported RBF 13
2.2 Wu-Wendland Functions 13

3 Problem Statement **15**

4 Literature Study **17**
4.1 Radial Basis function for SWES 17
4.2 RBF Generated Finite Differences 18
4.3 Scalable RBFFD 19
4.4 Standard Test Cases 19

5 Proposed methodology **23**
5.1 Reading coordinates from a NetCDF file 23

5.2 Atlas . . . 23
5.3 RBF and Operator Matrices - Python Implementation . . . 25
5.4 Shallow Water Equations . . . 26
5.5 Intialization and Simulation . . . 28

6 Mathematical Formulations **29**
6.1 Wu-Wendland Functions . . . 30
6.2 A Matrix and A inverse . . . 31
6.3 Projection Operator . . . 32
6.4 Shallow Water Equations - Cartesian Form . . . 32
6.5 Gradient Operators . . . 33
6.6 Discretized SWE using RBF . . . 34
6.7 Right Hand Side formulation . . . 35
6.8 Initialization . . . 36
6.8.1 Gravitational Constant . . . 36
6.8.2 Geopotential height of Shallow Water Equations . . . 36
6.8.3 Velocity Calculation . . . 37

7 Detailed implementation discussion **39**
7.1 Parallelization by Atlas . . . 40
7.1.1 Get coordinate information . . . 40
7.1.2 Creating `PointCloud` with Atlas features . . . 41
7.1.3 Find Neighborhood for RBF . . . 42
7.1.4 Construct RBF Operator Matrices . . . 43
7.1.5 Initialize Fields . . . 44
7.1.6 Solve PDEs using RK4 algorithm . . . 44
7.1.7 Visualize Results . . . 45

8 Experiments and Results **47**
8.1 Test Case 0 . . . 47
8.2 Test Case 2 . . . 48
8.3 Test Case 6 . . . 50

9 Conclusions **53**

10 Future prospects **55**
10.1 RBF Neural Network . . . 56

A Appendix **57**
A.1 Supporting libraries in Python . . . 57
A.2 Acronyms . . . 57
A.3 Complete python implementation . . . 58

Bibliography **85**

List of Figures

1.1 Atmospheric Sub Grid Scale Processes [3] 7
1.2 Biosphere Sub Grid Scale Processes[2] 8
1.3 Hydrosphere Sub Grid Scale Processes [24] 9

3.1 Node representation [28] 15

5.1 Equal Partitioning Algorithm [14] 24

8.1 Test case 0: Initial conditions 47
8.2 Test case 0: After 1 hr, time step size of 1200 seconds 48
8.3 Test case 2: Initial conditions 49
8.4 Test case 2: After 1 hr, time step size of 1200 seconds 49
8.5 Test case 6: Initial conditions 50
8.6 Test case 6: After 1 hr, time step size of 1200 seconds 50

Listings

7.1 Atlas initialize() and finalize() 40
7.2 Reading from NetCDF 40
7.3 Creating functionspace 41
7.4 Creating functionspace 42
7.5 Search Functionality 43
7.6 Halo Exchange 44
7.7 Gather 45
A.1 Atlas Installation 57
A.2 A_matrices 58
A.3 atlas_func_interface 60
A.4 atlas_main 61
A.5 construct_rhsd 68
A.6 helperfuncs 71
A.7 initializefields 74
A.8 operator_matrices 75
A.9 read_netcdf_file 77
A.10 testcases 78
A.11 visualization 80
A.12 WuWendland_functions 82

Chapter 1

Introduction

In the realm of global challenges facing humanity, climate change indisputably ranks among the top five. Worldwide initiatives are ongoing to curb the escalating global average temperature, projected to reach 2.6° Celsius by the century's end—a number surpassing the targeted 2-degree limit. [19]

Designated by the United Nations as the defining issue of our generation, climate change presents a pivotal era for humanity. From disruptive weather patterns jeopardizing food production to escalating sea levels intensifying the risk of catastrophic floods, its impacts are both global and unparalleled in scale. Immediate and drastic action is imperative today to facilitate adaptation for a better future, which, without intervention, promises to be arduous and costly.

Recognizing the urgency of intensified research in weather prediction and climate science, the requirement for skilled individuals to contribute to this domain has never been more important. As a devoted computational science and engineering student with a profound interest in computational fluid flow problems, I am motivated to contribute to the world of Numerical Weather Prediction (NWP) through this project.

This project constitutes a small attempt towards crafting improved solutions for prevailing issues. It revolves around advancing research into solving global atmospheric equations through the innovative application of mesh-free methods. The mesh-free methodology employed in this study uses a distinctive variant of compactly supported Radial Basis Functions(RBF).

1.1 Numerical Weather and Climate Prediction

1.1.1 History

The process of using numerical models for weather prediction has evolved over time tremendously. It began in 1920s through the ingenious work done by scientist, Lewis Fry Richardson who would calculate a 6 hour forecast by hand. [21] From calculating the equations by hand to predicting weather elements using Artificial Intelligence(AI), we have come a long way. It was not until the 1950s when computers were invented, resulting in a reduced computational time, that we could, for the first time, predict faster than the forecast period itself.

On a similar note, the first successful climate model dates back to 1956, when an American meteorologist named Norman A. Phillips [1] developed a mathematical model that could realistically depict monthly and seasonal weather patterns in troposphere. This was the first successful General Circulation Model (GCM) and became the foundation for all the future global geophysical models to be developed.

1.1.2 Classification

To delve a little deeper into numerical weather and climate prediction models, we must understand the various existing models and their domain of influence. Gradually, we will converge on our project topic and how it fits into the vast world of weather and climate prediction. We will explore traditional steps involved in NWP, before we talk about mesh-free Methods.

With over a century of knowledge and definitions about weather and climate, we can safely say that weather is the result of a collection of natural phenomena occurring in our atmosphere over a period of minutes, hours, days and at most, weeks. Whereas climate is described as weather patterns over a long period of time, for months and years at the least. The weather and climate prediction models are therefore developed uniquely.[12]

Prediction models are generally categorized based on their horizontal and/or vertical scale and the duration in future that it can realistically predict. Categorization based on geographical scale results in global and regional models.

Global models provide forecast output for the whole planet, typically for a couple of weeks into the future. Because they cover a bigger area, and a longer time duration, they're generally run at a lower resolution, both spatially (coarser grid) and temporally (less frequent forecast output). It is a weather forecasting model which initializes and forecasts the weather throughout the earth's troposphere and stratosphere.

Regional models on the other hand have a much higher resolution, but only provide forecast for a region of the planet, and only provide predictions about a couple of days out in time. The advantage of having higher resolution is that it lets regional models ”see” phenomena that the global models miss, like thunderstorms.

Categorizing based on the principle areas of application, we can divide NWP models into numerous sub categories: atmospheric, oceanic, air quality Models, Tropical Cyclone Models and the list is not exhaustive.

1.2 The Contributing Quad

Atmospheric Research has proven highly complicated owing to the fact that nature itself is extremely unpredictable and there are just too many variables contributing to the behavior of the prognostic variables. The 4 categories of physical domain give a high-level overview about the various elements at play while modeling for NWP are termed as The Quad - Atmosphere, Biosphere, Hydrosphere, Cryosphere.

Atmosphere - The entire air blanket enveloping the earth’s surface is called atmosphere along with its naturally occurring physical phenomena influencing wind movement, air temperature, pressure and humidity. The primary prognostic variables which form the basis of the primitive equations used in the present day prediction models are pressure, horizontal and vertical wind components, temperature, density, water vapour and turbulent kinetic energy.

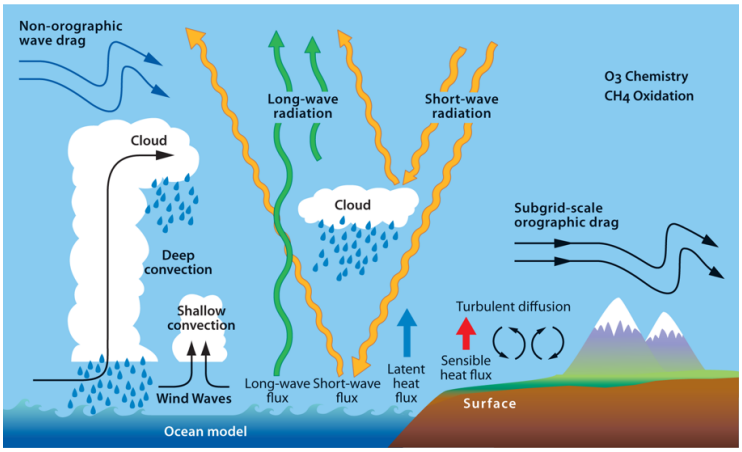


Figure 1.1: Atmospheric Sub Grid Scale Processes [3]

Biosphere - Biosphere includes all the biological and geological processes of earth along with its living species and the vegetation ecosystem. Studies

have shown that the inclusion of the effect of biophysical processes on the earth's surfaces significantly affect the prediction of meteorological variables, such as precipitation, low-level wind or sheltered temperature.

The main objective of surface topography used in NWP is to calculate the temperature and specific humidity at the lower limit of atmospheric models.

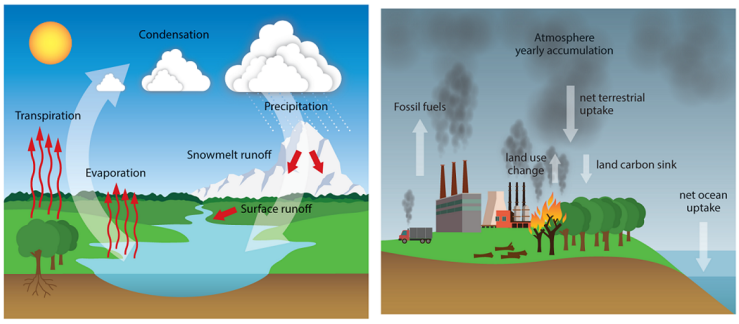


Figure 1.2: Biosphere Sub Grid Scale Processes[2]

Hydrosphere - The major water cycles involving water flows are precipitation, evaporation, and runoff. Apart from that, the water reservoirs on the surface of the earth can be divided into four categories: Soil (containing water in the root zone and often lower in the groundwater table, in liquid or ice form); Vegetation (crops,rain forests, grasslands etc); Snow and glaciers; Water reservoirs such as lakes and rivers.

Study of surface hydrological processes shows the exchange of water between surfaces and the lower atmosphere as well as water infiltration into the soil. First surface scheme was implemented in a general circulation model for climate study, in order to represent the water reservoir in the soil. This simple model had a single water reservoir, allowing the evaporation of continental surfaces to be controlled according to their water percentage. These schemes later evolved into two-layer or multi-layer models with finer vertical discretization.

Infiltration is often represented according to Darcy's law [8] which is generalized for unsaturated soils with several developments from the watershed hydrology study. This allows the development and use of soil models, as in the case of the ECMWF , coupled with river routing models in flood forecasting, as in the global flood awareness system project.

The oceans are a major source of heat and momentum exchange with the atmosphere. They have a constant influence on the NWP models at all times. Heat exchange with the atmosphere is the result of the surface temperature of the seas, which is influenced by weather conditions but also by ocean currents (e.g. Gulf Stream and Kuroshio in the northern hemisphere), and by climatic

large scale phenomena such as ENSO (El Niño Southern Oscillation), with strong consequences on atmospheric circulation at regional and continental scales.

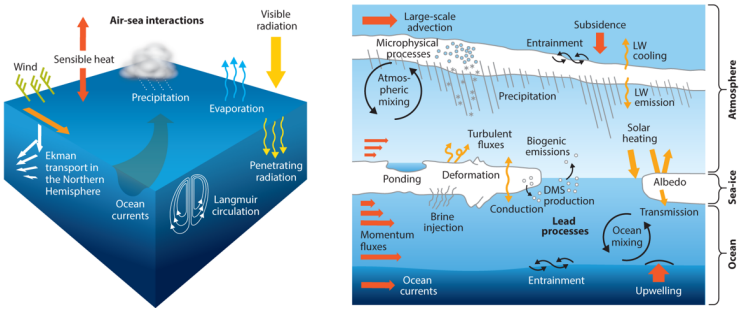


Figure 1.3: Hydrosphere Sub Grid Scale Processes [24]

Cryosphere - Snow and ice (marine and continental) are important energy pockets that can directly affect the rate of warming or cooling of the atmosphere and ground by releasing or absorbing latent heat. The snow pack can act as an effective thermal insulator, as it can contain up to 90 percent air (for a density of about 100 kg/m3 in the case of freshly deposited snow). Snow also is a key contributor of fresh water that modulates rivers, especially at high latitudes.

In high latitude and mountainous areas, underlying snow acts as an important reservoir of energy and water. Therefore, an accurate representation of snow mass in the NWP model is crucial for all forecast times, with direct consequences for hydrology and prediction of temperatures.[12]

1.3 Mesh-free Methods

Contrary to typical numerical methods that use a mesh, mesh-free methods are a type of methods which do not require a connection between different nodes of the domain. Rather, they function on the principle of the interaction of a node with its neighboring nodes. Hence, these type of methods discard the concepts of mesh edges, surfaces or volume centers.[18]

Traditionally, Finite Difference Method (FDM) and Finite Volume Method(FVM) were defined on a mesh of grid points. Each point has a predefined set of neighbors, and this interdependent set of neighboring points is called a stencil. All the mathematical operators are defined on this stencil, which are eventually used to construct equations for simulation.

Mesh-free methods have two significant merits over mesh methods. Traditional structured grids which form a mesh in FDM, FVM methods are

generally not suitable for spherical geometry, because the vertical lines in a mesh may tend to converge near the poles, leading to singularities in the mathematical operators dependant on them. Hence, mesh-free methods are more useful when we have a spherical type of geometry, because the points are uniformly distributed even near the poles. Secondly, parallelization is much more convenient to implement in mesh-free methods, compared to the mesh methods.

Although there are a wide variety of mesh-free methods, for this project we will focus on Radial Basis Functions(RBF). RBFs are widely popular for achieving spectral accuracy in multi-dimensions for extremely arbitrary geometry. They provide solutions with astounding algorithmic simplicity.

RBF present very strong characteristic advantages in the context of solving PDE. We will look at the merits of each of them.

1. Mesh-free Method: RBF, being a mesh-free method, is independent of the complexities of the domain geometry. It gives freedom to attempt solutions for otherwise difficult problem statements for mesh methods (FDM, FVM, FEM)
2. Algorithmic Simplicity: Since, the function domain can be expressed in the form of an interpolation matrix, the final solution can be obtained by solving an explicit system of equations through already established solving methodologies.
3. Robustness (Large Time Steps): We can choose the number of interacting neighboring nodes for a specific node by varying the radius of the RBF. Since one node derives values from more than just its immediate neighbors, we can experiment with larger time steps satisfying the CFL(Courant-Friedrichs-Lewy) condition.
4. Spectral Accuracy: RBF (only global) can be very effective for scattered data in any number of dimensions. It gives accurate results with high orders of convergence with increase in the number of bases for special cases of global domain.

Chapter 2

Radial Basis Functions

The first known Radial Basis Function (RBF) dates back to 1930s where a Hungarian mathematician, Marcel Riesz, introduced the topic in the context of functional analysis. In the second half of 20th century, the application of RBF was explored for interpolation, approximation and numerical methods. The latter application area is of interest to us. Particularly, the use of RBF in mesh-free methods to solve Partial Differential Equations (PDE). The first attempt at such a formulation was achieved in the Kansa Method, explained in detail in the subsequent section. [6]

To begin with, RBF is a real valued function φ which depends only on the distance between a fixed node point (eg. $\mathbf{x_1}$)and another point ($\mathbf{x}$) provided as input. Any function that satisfies the property

$$\varphi(\mathbf{x}) = \hat{\varphi}(\|\mathbf{x_1} - \mathbf{x}\|) \tag{2.1}$$

is a radial function, where the l_2 norm can be defined as

$$\|\mathbf{x_1} - \mathbf{x}\| = \sqrt{(x_1 - x)^2 + (y_1 - y)^2 + (z_1 - z)^2} \tag{2.2}$$

A collection of these functions φ_k for the nodes in the range $1, \ldots, n$, form a basis to approximate another function space. This collection of functions is called RBF. They can be collocated in an interpolation matrix show in equation (2.3)

$$\begin{bmatrix} \varphi(\|\mathbf{x}_1 - \mathbf{x}_1\|) & \varphi(\|\mathbf{x}_2 - \mathbf{x}_1\|) & \ldots & \varphi(\|\mathbf{x}_n - \mathbf{x}_1\|) \\ \varphi(\|\mathbf{x}_1 - \mathbf{x}_2\|) & \varphi(\|\mathbf{x}_2 - \mathbf{x}_2\|) & \ldots & \varphi(\|\mathbf{x}_n - \mathbf{x}_2\|) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi(\|\mathbf{x}_1 - \mathbf{x}_n\|) & \varphi(\|\mathbf{x}_2 - \mathbf{x}_n\|) & \ldots & \varphi(\|\mathbf{x}_n - \mathbf{x}_n\|) \end{bmatrix} \tag{2.3}$$

First RBF based numerical method was developed by E. J. Kansa in early 1990s to solve the elliptic Poisson's equation and the linear advection-diffusion equation. Popularized as the Kansa method, it is also known as modified Multi-quadratic (MQ) scheme or MQ collocation method. [20]

The Kansa method approximates the function value at each node by a linear combination of the RBF

$$U(X) = \sum_{i=1}^{N} \alpha_i \psi(r_i) \tag{2.4}$$

where α_i are the coefficients to be evaluated. $\psi(r_i)$ represents the RBF.

2.1 Classification

Commonly used RBFs employ the Euclidean norm for calculating the inter node distances. Another parameter which appears quite often in formulations is the shape parameter ε. It finds its use in scaling the input provided to the radial Kernel. Although, there is a standardized method to calculate distance between two nodes, we have the freedom to choose the functions which form the basis of RBF.[15]

2.1.1 Infinitely Smooth RBF

Some examples of infinitely smooth radial basis functions are given below. Each of them are positive definite functions that use a shape parameter ε to manipulate the input values to the RBF kernel.

- Gaussian
$$\psi(r) = e^{-(\varepsilon r)^2} \tag{2.5}$$

- Multi-Quadratic (MQ)
$$\psi(r) = \sqrt{1 + (\varepsilon r)^2} \tag{2.6}$$

- Inverse Quadratic
$$\psi(r) = \frac{1}{1 + (\varepsilon r)^2} \tag{2.7}$$

- Inverse Multi-Quadratic
$$\psi(r) = \frac{1}{\sqrt{1 + (\varepsilon r)^2}} \tag{2.8}$$

2.1.2 Polyharmonic Splines

Polyharmonic Splines (PHS) are an excellent tool for interpolation. Although, it is critical to ensure that $r \neq 0$, as $ln(r) = \text{inf}$ when r tends to 0

$$\varphi(r) = r^k \text{for } k = 1, 3, 5, \ldots \tag{2.9}$$

$$\varphi(r) = r^k \ln(r) \text{for } k = 2, 4, 6, \ldots \tag{2.10}$$

2.1.3 Thin plate spline

These are a special case of PHS. They are defined as follows.

$$\varphi(r) = r^2 \ln(r) \tag{2.11}$$

2.1.4 Compactly supported RBF

Our favourite type of RBF, which was used in the project are the compactly supported RBF. The special cases of compact support RBF will be covered in detail in the section 2.2

$$\varphi(r) = \begin{cases} \exp\left(-\frac{1}{1-(\varepsilon r)^2}\right) & \text{for } r < \frac{1}{\varepsilon} \\ 0 & \text{otherwise} \end{cases} \tag{2.12}$$

2.2 Wu-Wendland Functions

Wendland Functions are special type of compactly supported RBFs, which were first introduced and studied by Wu [31] and formalized by Wendland in 1995. Therefore, we shall refer to them as Wu-Wendland functions through out this paper. They are constructed by repeatedly applying an integral operator to the Askey's truncated power functions. [32]

Askey's truncated power function can be defined as follows:

$$\varphi_l(x) = (1 - \|x\|_2)^l{}_+ = \begin{cases} (1 - \|x\|_2)^l & \text{for } 1 - \|x\|_2 > 0 \\ 0 & \text{for } 1 - \|x\|_2 \leq 0 \end{cases} \tag{2.13}$$

Now, let us consider this integral operator for Wendland Functions:

$$(I\varphi)(r) := \int_r^\infty t\varphi(t)\, dt \text{ for } r \geq 0 \tag{2.14}$$

We can define Wendland functions as [29]

$$\varphi_{d,k}(r) = I^s \varphi_l(r) \tag{2.15}$$

$l = \lfloor \frac{d}{2} \rfloor + s + 1$, $\varphi_l = (1-r)^l_+$, s being the smoothness parameter and d representing the dimensions of the domain.

This fundamental definition has been used in this project for constructing the RBF matrices. Specific formulation is illustrated in chapter 6.

Chapter 3

Problem Statement

The problem statement is to develop a parallel algorithm to formulate approximate numerical operators for 2D and 3D PDEs used in atmospheric dynamics in a spherical geometry by employing a very special category of mesh-free methods called compact supported Radial Basis Function (RBF) or Wu-Wendland functions.

An extremely ideal case of atmospheric dynamics is used to test the framework. Earth's surface is assumed to be smooth and the entire geometry is scaled to a unit sphere. An icosahedral grid is used for the calculations, because of its semi-uniformly spaced node distribution. The following picture of nodes on a sphere is just for visualization purpose. It belongs to a different grid type and is not derived from this project.

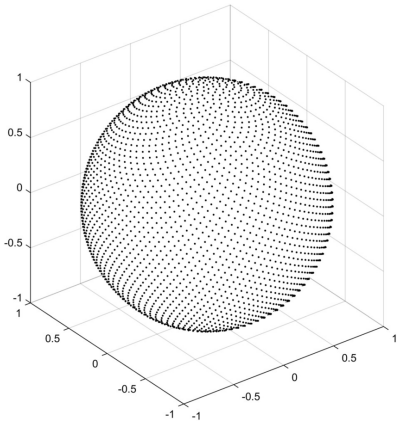


Figure 3.1: Node representation [28]

Shallow Water Equations (SWE) would be a simple testing tool to evaluate the algorithmic development of mesh-free numerical operators of 2D and 3D

PDEs used to simulate atmospheric dynamics.

Main objectives of this thesis project are listed below:

- To develop a completely parallel implementation in Python by integrating powerful libraries built for efficient performance on a supercomputer node.
- To use a mesh-free method, Wu-Wendland functions, as a discretization technique for the formulation of operator approximations.
- To test the capability of an industrial library for its efficiency in working on high resolution unstructured grids on a 2D and 3D spherical geometry

The following chapter will highlight the prior research done on this topic and the inspirations drawn for this project.

Chapter 4

Literature Study

In this chapter we will explore some excellent pieces of research pertaining to our topic. These papers became the primary inspiration for my thesis project.

4.1 Radial Basis function for SWES

Title: A Radial Basis Function Method for the Shallow Water Equations on a Sphere [16]

At the time of this paper, RBFs were only getting popular for the previous 15 years for their application in solving PDE as part of numerical methods. It traces the motivations for using RBF methodology as a solution for interpolating sparse scattered data on a sphere to a question asked by R.L. Hardy. 'Given a set of sparse scattered data, $f_{j=1}^{N}$ at the node locations $X_{j=1}^{N} \subset R^d$, can an interpolant be constructed that adequately represents the unknown surface?' Hardy's novel approach suggested to construct the interpolant from linear combinations of a single basis function which is radially symmetric about its center and with a direct dependency on node locations.

This paper also uses a shape parameter ε to manipulate the smoothness of RBF. It states that an infinitely smooth RBF depends on the shape parameter. In 1-D when the shape parameter tends to zero (Flat RBF), the interpolation produces pseudo-spectral methods if the nodes are appropriately placed.

It further explains that RBF approximation can be improved by increasing the number of terms in the expansion and/or decreasing the shape parameter ε.

In order to perform a study with regards to its spatial and temporal errors, two nonlinear test cases with known analytical solutions are considered. The first is global steady-state flow with a compactly-supported velocity field while the second is unsteady flow where features in the flow must be kept intact without dispersion.

Error and time stability studies are both performed as the number of nodes is uniformly increased. The shape parameter of the RBF is varied, especially in the flat basis function limit.

A very unique approach that this paper presents is the use of Cartesian coordinate system, instead of spherical or longitude-latitude coordinate system, which is norm for models developed for a sphere. This approach was chosen to avoid the problem of singularity near the poles, that spherical coordinate system brings when applied to the RBF. This is a key lesson and this methodology has been implemented in this thesis project as well.

Results of this paper show that the global RBF method is spectral, giving exceptionally high accuracy for low number of basis functions while being able to take unusually large time-steps. In order to put it in the context of other commonly used global spectral methods on a sphere, comparisons were given with respect to spherical harmonics, double Fourier series, and spectral element methods.

4.2 RBF Generated Finite Differences

Title: Radial Basis Function-Generated Finite Differences: A mesh-Free Method for Computational Geosciences [17]

This paper develops more on the concept of Radial basis function-generated finite differences (RBF-FD). RBF-FD are a branch of global RBF. If a particular domain has N number of nodes, a global RBF interpolation matrix includes all the points. In the case of RBF-FD, only $n \leq N$ neighboring nodes are chosen for each stencil constructed for calculations at each node. RBF-FD uses local approximations for derivatives instead of global ones, which are 99 percent empty. At the sacrifice of spectral accuracy provided by the global RBF, much is gained in terms of memory usage and runtime.

This paper touches upon various aspects of RBF-FD and concepts needed to understand the new approach. The concepts are explained using distinct applications on spherical geometry. Shallow water equations on a sphere, reaction–diffusion equations, global electric circuit, and mantle convection in a spherical shell are covered as part of the research. The results from the last three of these applications are new and have not been presented before for RBF-FD, when this paper was published.

Apart from shedding light on the RBF-FD approach, this paper discusses various computational options available to compute the results.

- The Contour-Padé Algorithm
- The RBF-QR Algorithm
- The RBF-GA Algorithm

The ill-conditioning of the RBF interpolation matrix is also discussed in great detail. Considerable number of benchmarks are performed and tabulated for the validation of results when compared to the results obtained using other discretization methods.

4.3 Scalable RBFFD

Title : A scalable RBF–FD method for atmospheric flow [25]

RBF-FD have been shown to outperform established pseudo-spectral and discontinuous Galerkin methods for shallow water test problems. In order to be competitive for very large scale simulations, the RBF–FD methods needs to be efficiently implemented for modern multicore based computer architectures.

This paper takes the concept of RBF-FD described in Section 4.2 and develops a MATLAB algorithm for its implementation on a large scale. It employs task based parallel programming method to parallelize the Shallow Water Equation (SWE) solver. The parallelization was performed using the SuperGlue library for task-based parallel programming.

It goes in length to describe why a task-based parallelization is a suitable approach for productivity and performance. A task-based parallelization was chosen for two main reasons:

- Programmer productivity: A key idea in task parallel programming is that it allows the programmer to write a sequential code in terms of computational tasks, and then a run-time system handles the parallel execution of the tasks.
- Performance: Tasks are scheduled dynamically at run-time to the cores, processors, and computational nodes. This ensures minimal idle time for the computing machine.

Finally, results were presented for global shallow water benchmark problems with a 30 km resolution. The main metric for success was determined by the speedup obtained using parallel techniques.

4.4 Standard Test Cases

Title: A Standard Test Set for Numerical Approximations to the Shallow Water Equations in Spherical Geometry [30]

This paper proposes a set of 7 test cases for the evaluation of numerical approximation methods used for the solution of SWE on a spherical geometry. These test cases highlight the potential trade-offs that a scientist may have to face while modeling for weather and climate prediction.

Before an algorithm is applied to the full atmospheric model, it must perform well on these standard tests. These test cases are provided with increasing levels of complexity. One of the test cases is specific for testing computer performance or the efficiency of an algorithm adapted for functioning on massively parallel computers.

In this project, we are exploring the results obtained in test cases: 2,6 and 0 (which is very similar to test case 1) [23].

Test case 0: Arctic pudding initially at rest

A cosine bell of mass is initialized near the north pole on the unit sphere. Initially, the system is at rest with no Coriolis force. The height field is initialized as per equation (4.1)

$$h(\lambda,\theta) = \begin{cases} (h_\circ/2)(1+\cos(\frac{\pi r}{R})) & \text{if } r < R \\ 0 & \text{otherwise} \end{cases} \tag{4.1}$$

the parameters used in the above equation are scaled for the calculations on the unit sphere. They are defined as follows:

$$\begin{aligned} h_\circ &= \frac{1000}{6371000} = 1.569 \times 10^{-4}[L^1] \\ a &= 1.0[L^1] \text{ (radius of the sphere)} \\ R &= \frac{a}{3}[L^1] \end{aligned}$$

r is the great circle distance between any node to the center of the pudding which is north pole. The coordinates of the center at north pole are defined by (λ_c, θ_c)

$$\begin{aligned} (\lambda_c, \theta_c) &= (0, \pi/2) \\ r &= a \arccos\left(\sin\theta_c \sin\theta + \cos\theta_c \cos\theta \cos\left(\lambda - \lambda_c\right)\right) \end{aligned}$$

Test case 2: Global Steady State Nonlinear Zonal Geostrophic Flow

It involves a steady-state solution to the non-linear shallow water equations, with characteristics such as solid body rotation or zonal flow. The geopotential height field is associated with this rotation, and the Coriolis parameter varies with latitude and longitude, allowing for the specification of flow in spherical coordinates.

The test case explores different configurations or scenarios, involving variations in the orientation of the flow with respect to the Earth's rotation axis.

Some constants according to the test case 2 are defined as follows.

$$
\begin{aligned}
&g = 1.5454 \times 10^{-8}[L^1T^{-2}] \\
&radius = a = 1.0[L^1] \\
&u_\circ = 2 \times \pi \times a/(12 \times 86400)[L^1T^{-1}] \\
&\alpha = 0.0 \\
&h_\circ = 2.94 \times 10^4/(g \times 6371000^2); \\
&\Omega = 7.292 \times 10^{-5}[T^{-1}]
\end{aligned}
$$

The zonal (u) and meridional (v) velocity fields are given by the following equations

$$u = u_\circ(\cos\theta\cos\alpha + \cos\lambda\sin\theta\sin\alpha) \tag{4.2}$$

$$v = -u_\circ\sin\theta\sin\alpha \tag{4.3}$$

and the analytical geo-potential height is given by

$$gh = gh_\circ - \left(a\Omega u_\circ + \frac{u_\circ^2}{2}\right)\cdot(-\cos\lambda\cos\theta\sin\alpha + \sin\theta\cos\alpha)^2 \tag{4.4}$$

Test case 6: Rossby-Haurwitz Wave

Rossby-Haurwitz waves are important solutions in geophysical fluid dynamics, particularly in the context of atmospheric and oceanic motions. These waves serve as analytic solutions to the nonlinear barotropic vorticity equation on a sphere. They have been widely adopted as standard test cases in meteorological simulations. Some constants and parameters for the initialization of velocity and height fields according to the test case 6 are defined by the following equations. They are scaled for the calc

$$
\begin{aligned}
g &= 1.5454 \times 10^{-8}[L^1T^{-2}] \\
\Omega &= 7.292 \times 10^{-5}[T^{-1}] \\
\omega &= 7.848 \times 10^{-6}[T^{-1}] \\
K &= \omega \\
h_\circ &= 1.25568 \times 10^{-3} \\
R &= 4 \\
radius &= a = 1.0[L^1]
\end{aligned}
$$

$$u = a\omega\cos(\theta) + aK\cos^{R-1}(\theta)(R\sin^2(\theta) - \cos^2(\theta))\cos(R\lambda) \tag{4.5}$$

$$v = -aKR\cos^{R-1}(\theta)\sin(\theta)\sin(R\lambda) \tag{4.6}$$

$$gh = gh_\circ + a^2A(\theta) + a^2B(\theta)\cos(R\lambda) + a^2C(\theta)\cos(2R\lambda) \tag{4.7}$$

$$
\begin{aligned}
A(\theta) = {} & \frac{\omega}{2}(2\Omega + \omega)\cos^2(\theta) + \\
& \frac{K^2}{4}\cos^{2R}(\theta)\left[(R+1)\cos^2(\theta) + (2R^2 - R - 2) - 2R^2\cos^{-2}(\theta)\right]
\end{aligned}
$$

$$B(\theta) = \frac{2(\Omega + \omega)K}{(R+1)(R+2)}\cos^R(\theta)\left[(R^2 + 2R + 2) - (R+1)^2\cos^2(\theta)\right]$$

$$C(\theta) = \frac{1}{4}K^2\cos^{2R}(\theta)\left[(R+1)\cos^2(\theta) - (R+2)\right]$$

The papers discussed so far were the primary inspiration for the development of this project. The upcoming chapters discuss the proposed methodology, mathematical formulations involved and Python implementation in detail. Lastly, we will discuss results and conclude the topic.

Chapter 5

Proposed methodology

5.1 Reading coordinates from a NetCDF file

NetCDF is a file format, preferred by industry leaders, for storing multidimensional scientific data (variables) such as temperature, humidity, pressure, wind speed, and direction. Huge data sets are easily stored and managed through simple access functions. For this project, a NetCDF file containing coordinate information in latitude and longitude format was used. It was provided by the thesis advisor from one of the CSCS grid repositories. The grid has a semi-uniform distribution of 20,480 nodes. The latitude and longitude information is read into a 2D Matrix for further use.

5.2 Atlas

"Atlas is an ECMWF library for parallel data-structures, supporting unstructured grids and function spaces, with the aim to investigate alternative more scalable dynamical core options for Earth System models, and to support modern interpolation and product generation software". [10]

This project needs the creation of a 3D unstructured grid, which resembles a `Pointcloud` of uniformly distributed nodes on a spherical geometry. Atlas provides the functionality to create a 2D unstructured grid, but it was not sufficient for this mesh-free project. The existing functionality was extended by an expert scientist from ECMWF to support the 3D unstructured nature of our problem statement.

Atlas employs various strategies to partition a grid based on the use cases. Each strategy may offer different advantages depending on the grid and numerical algorithm to be used. The strategies existing in the Atlas library are: Checkerboard, EqualRegions, MatchingMesh and MatchingFunctionspace. [14]

EqualRegions partitioning is used in our case. The partitioning algorithm divides a two-dimensional grid of a sphere into bands starting from the North Pole (NP) and all the way till South Pole (SP). These bands are arranged in zonal directions and each band is further divided into regions containing equal number of grid points. The only exception comes for the grid points around NP and SP, where we get the NP and SP caps. The following image is a representative picture for the equal partition division of a classic Gaussian grid.

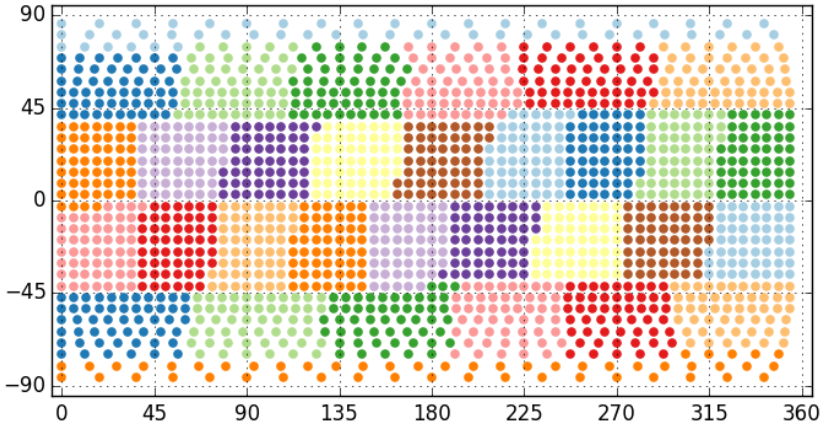


Figure 5.1: Equal Partitioning Algorithm [14]

The new `Pointcloud` functionality has the following key features which are used in this project.

- `functionspace`: Creates a decomposed `functionspace` by taking the list of coordinates and halo radius as an input. It creates a point cloud of nodes distributed in a spherical geometry.
- Creating and accessing fields: Allows to create fields linked to the created function space. Variable data can be stored during the computation and fields can be accessed as numpy arrays.
- Finding neighborhood: Given a specific radius, the tool finds the closest neighbors of each node and returns their global and local indices for

each subdomain. It employs k-d tree algorithm to find the list of closest nodes.

- Halo exchange: It provides the functionality to perform halo exchange on the borders of each subdomain to synchronize the values of the fields with other parallel processes.
- Gather: A gathering functionality to gather the field data from across all processes. This is essential for final plotting and visualization of the results.

This paper will become the first known documentation of the Atlas's Point-cloud Functionality for 3D unstructured grids.The syntax of the commands and the specific functions could be found in the chapter 7

5.3 RBF and Operator Matrices - Python Implementation

Once we have a `functionspace` decomposed into subdomains based on our unstructured 3D grid and a list of nearest indices for each node, we can proceed to create RBF interpolation matrices. Matrix A, formulation explained in the next chapter, represents the interpolation matrix collocated with Wu-Wendland Functions.

$$\begin{bmatrix} \varphi(\|\mathbf{x}_1 - \mathbf{x}_1\|) & \varphi(\|\mathbf{x}_2 - \mathbf{x}_1\|) & \dots & \varphi(\|\mathbf{x}_n - \mathbf{x}_1\|) \\ \varphi(\|\mathbf{x}_1 - \mathbf{x}_2\|) & \varphi(\|\mathbf{x}_2 - \mathbf{x}_2\|) & \dots & \varphi(\|\mathbf{x}_n - \mathbf{x}_2\|) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi(\|\mathbf{x}_1 - \mathbf{x}_n\|) & \varphi(\|\mathbf{x}_2 - \mathbf{x}_n\|) & \dots & \varphi(\|\mathbf{x}_n - \mathbf{x}_n\|) \end{bmatrix} \tag{5.1}$$

These A Matrices are created for each node inside the subdomain. The LU decomposition of these A Matrices is calculated using BLAS (Basic Linear Algebra Subprograms) functionalities in Python. The inverse is used in the construction of gradient operators for the chosen RBF.

Let us consider a function defining the behavior of some atmospheric variables as an RBF expansion.

$$f(\mathbf{x}) = \sum_{k=1}^{N} c_k \varphi_k(r(\mathbf{x})) \tag{5.2}$$

When the projected gradient operator is applied to the function in x direction, calculating it for all the nodes in the domain gives a formulation as follows:

$$[p_x \cdot \nabla f(\mathbf{x})]\Big|_{\mathbf{x}=\mathbf{x_j}} = \sum_{k=1}^{N} c_k [p_x \cdot \nabla \varphi_k(r(\mathbf{x}))]\Big|_{\mathbf{x}=\mathbf{x_j}}, \quad j = 1, \ldots, N \tag{5.3}$$

$$\begin{aligned} &= \sum_{k=1}^{N} c_k \left(x_j \mathbf{x_j^T x_k} - x_k \right) \frac{\varphi_k'(r(\mathbf{x_j}))}{r(\mathbf{x_j})} \quad j = 1, \ldots, N \\ &= B^x c \\ &= (B^x A^{-1}) f \\ &= D^x{}_N f \end{aligned}$$

This $D^x{}_N$ represents an array of our gradient operators for each node in the x direction. When all the operators for each node are collocated together, they form the gradient operator matrix.

The gradient operators, thus constructed, are applied to the Shallow Water Equation (SWE) terms to get the discretized form of the PDE. Consequently, we develop the right hand side of the partial derivative terms of the SWE.

5.4 Shallow Water Equations

Shallow Water Equations (SWE) are a set of hyperbolic partial differential equations that describe a thin layer of fluid of constant density in a hydrostatic balance bounded from below by a topography and from above a free surface. [27]

They are derived by depth integrating the Navier Stokes Equations where the vertical dynamics can be neglected in comparison to the horizontal effects.

SWE have a wide range of applications. We can learn about some of the popular applications:

- Tsunami Prediction
- Atmospheric Flows
- Storm Surges
- Flows around a structure
- Dam Breaks
- Planetary Flows

Again, as previously established that no amount of research is exhaustive when it comes to Nature, the above list also represents only a small subset

of the entire range of applications. The general advective form of Shallow Water Equation on a spherical geometry in spherical coordinate system can be described as follows:

$$\frac{\partial h}{\partial t} + \nabla \cdot (\mathbf{u}h) = 0$$
$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla)\mathbf{u} + f\mathbf{k} \times \mathbf{u} = -g\nabla h$$

Where:

- h is the fluid height,
- $\mathbf{u}$ is the velocity vector, describing the horizontal motion of the fluid.
- t is time.
- ∇ is the gradient operator, indicating spatial variation.
- f is the Coriolis parameter, representing the effect of the Earth's rotation.
- $\times$ denotes the cross product, used for the Coriolis force.
- $\mathbf{k}$ is the unit vector in the vertical direction.
- g is the acceleration due to gravity, influencing the fluid motion.

A key misconception that might arise for people learning about SWE would be the application, because of the name. Here the water would translate to any fluid, not just water. It could be air and hence, SWE are widely accepted as a tool to test PDE used for atmospheric dynamics close to the surface of the earth.

5.5 Intialization and Simulation

We will use test cases presented in the paper published in 1992 [30] for the validation of our algorithm. Fields are initialized with the appropriate test case data.

Then we will compute the next time step values in an explicit Runge-Kutta 4th order time stepping loop. There will be a halo exchange performed after the calculation of each internal step. These methods are a popular choice for temporal descretization problems in Numerical Analysis. Specifically, the 4th order is the most widely used class of RK methods and it has proven to give very good results.

Let us recap the general idea of an RK4 algorithm.

Algorithm 1 Runge-Kutta 4th Order for Time-Dependent PDE

procedure RK4(Initial condition, time step, final time)
  Set t to the initial time
  while $t <$ final time **do**
    Compute k_1 using the PDE and the current solution (initial condition for first iteration)
    Compute k_2 using the PDE and the solution at $t + \frac{\Delta t}{2}$ with k_1
    Compute k_3 using the PDE and the solution at $t + \frac{\Delta t}{2}$ with k_2
    Compute k_4 using the PDE and the solution at $t + \Delta t$ with k_3
    Update the solution: $f(t + \Delta t) = f(t) + \frac{1}{6}(k_1 + 2k_2 + 2k_3 + k_4)$
    Update t: $t = t + \Delta t$
  end while
end procedure

Finally, the results are visualized using matplotlib library functions.

Chapter 6

Mathematical Formulations

Let us understand the mathematical formulations used in this project. They vary slightly from the standard form available in the literature. Since the entire algorithm is parallelized, we need to keep in mind that, all the formulations refer to the elements of the decomposed subdomain. We will refer to the latitude-longitude points from the NetCDF file, as described in section 5.1, as nodes. As mentioned in the problem statement, the earth's dimensions are scaled to a unit sphere.

Let us define some nomenclature and terms used in the project for mathematical calculations.

- j : index to denote individual nodes in the subdomain.
- p : number of decomposed subdomains
- n_p : number of nodes in each decomposed domain
- R : primary radius which forms as the input for the Radial Basis Functions (RBF)
- n_j^R : the number of nodes within a radius R located around each node j. This cluster of nodes form the neighborhood at j.
- k : index to denote the nodes inside the neighborhood.
- $\mathbf{x_j}$: The bold lettering denotes the $[1,3]$ row vector containing the Cartesian coordinate information at node j.
- x_j : The regular lower case letter denotes the $[1,1]$ scalar containing the x coordinate information at j. Similarly for y and z.

6.1 Wu-Wendland Functions

Wu-Wendland functions are a special category of compactly supported Radial Basis Functions. From our project's context, the compactness of the support is limited to the neighborhood of nodes obtained for each node j

We will use the standard definition of Wu-Wendland functions as defined in the equations 2.13 and 2.15 and modify it for our formulation.

$$\varphi_l(r) = (1-r)^l{}_+ = \begin{cases} (1-r)^l & \text{for } 1 \geq r \\ 0 & \text{for else} \end{cases} \tag{6.1}$$

Now, as we can see from the above definition, our function φ depends on multiple variables: l and r

r is the Eucledian norm between the current node j and any other node k in the neighborhood.

$$r = \|\mathbf{x_j} - \mathbf{x_k}\|_2 \tag{6.2}$$

l defines the order of the Wu-Wendland Functions. It depends on two more variables d and k as defined below.

$$l = \lfloor \frac{d}{2} \rfloor + s + 1 \tag{6.3}$$

d is the dimension of the domain, in our case it is 3
s is the smoothness parameter, increasing it will get us higher orders of Wu-Wendland Functions.

Assuming $d = 3$, let us tabulate different values of l for different values of s

Smoothness Factor s	Order of function, $l = \lfloor \frac{3}{2} \rfloor + s + 1$
$s = 0$	$l = 2$
$s = 1$	$l = 3$
$s = 2$	$l = 4$
$s = 3$	$l = 5$

Table 6.1: l,s values

These Wu-Wendland formulations are defined in the Python implementation. User is allowed to choose the order of the Wu-Wendland functions by changing function call statement.

Similarly, the derivatives of these Wu-Wendland functions are used in the formulation of gradient operator matrix which will be covered in section 6.5

Values of l	Wu-Wendland Formulation
2	$\varphi_l(r) = (1-r)^l$
3	$\varphi_l(r) = (1-r)^{l+1}((l+1)r+1)$
4	$\varphi_l(r) = (1-r)^{l+2}((l^2+4l+3)r^2+(3l+6)r+3)$
5	$\varphi_l(r) = (1-r)^{l+3}((l^3+9l^2+23l+15)r^3+(6l^2+36l+45)r^2+(15l+45)r+15)$

Table 6.2: Wu-Wendland functions

6.2 A Matrix and A inverse

A matrix is the interpolation matrix which collocates all the Wu-Wendland function corresponding to one neighborhood cluster. It is a square symmetric matrix of size $[n_j^R, n_j^R]$. The formulation of the A matrix is similar to the equation 2.3

$$\begin{bmatrix} \varphi(\|\mathbf{x}_1-\mathbf{x}_1\|) & \varphi(\|\mathbf{x}_2-\mathbf{x}_1\|) & \dots & \varphi(\|\mathbf{x}_{n_j^R}-\mathbf{x}_1\|) \\ \varphi(\|\mathbf{x}_1-\mathbf{x}_2\|) & \varphi(\|\mathbf{x}_2-\mathbf{x}_2\|) & \dots & \varphi(\|\mathbf{x}_{n_j^R}-\mathbf{x}_2\|) \\ \vdots & \vdots & \ddots & \vdots \\ \varphi(\|\mathbf{x}_1-\mathbf{x}_{n_j^R}\|) & \varphi(\|\mathbf{x}_2-\mathbf{x}_{n_j^R}\|) & \dots & \varphi(\|\mathbf{x}_{n_j^R}-\mathbf{x}_{n_j^R}\|) \end{bmatrix} \tag{6.4}$$

According to Linear Algebraic Methods, a Lower-Upper(LU) factorization of a matrix forms the matrix as a product of lower and upper triangular matrices. This is obtained by applying permutations on rows and columns to convert the square matrix into two triangular matrices: Lower and Upper.

Since, A is a dense symmetric matrix. We can use LU factorization for calculating the inverse of the matrix. The linear algebraic method describes the calculation of the inverse of a square matrix using LU factorization in the following steps.

1. A is operated upon by a series of row and column permutations to transform it into an upper triangular matrix, by introducing zeros below the diagonal.
$$L_n L_{n-1} \dots L_2 L_1 A = U \tag{6.5}$$

2. These consecutive operators when collocated together form a unit lower triangular matrix $L = L_n L_{n-1} \dots L_2 L_1$

$$LA = U \tag{6.6}$$

3. If we denote L as L^{-1} to take the lower triangular matrix to the right hand side of the equation, we get the general form of LU factorization.

$$L^{-1}A = U \tag{6.7}$$

$$A = L.U \tag{6.8}$$

But this algorithm is unstable in its purest form. Pivoting , either partial or complete, introduces the stability in algorithm. It is a series of row and column permutations which organize the elements in the matrix for ideal factorization.

SciPy is a free and open source library for scientific computing and numerical analysis in Python. It offers modules in linear algebra which contains a function which can calculate LU factorization of square matrices using partial pivoting. This Python routine was used for calculating the inverse of A matrix. [7]

6.3 Projection Operator

We need a projection operators to confine our PDEs to the surface of the unit sphere. This is done by projecting the 3D Cartesian vectors onto a plane tangent to the sphere. If our sphere is already defined in Cartesian coordinates (x, y, z) , it is easy to construct the operator, as the normal is at (x, y, z) is just its position vector $X = x, y, z^T$.

Hence, if $\mathbf{u}$ is a vector at X, then $\mathbf{u} - XX^T\mathbf{u}$ is the projection of $\mathbf{u}$ on to the plane tangent to the unit sphere at X. Therefore, we can define the Projection operator P as a matrix:

$$P = I - xx^T = \begin{bmatrix} (1-x^2) & -xy & -xz \\ -xy & (1-y^2) & -yz \\ -xz & -yz & (1-z^2) \end{bmatrix} = \begin{bmatrix} \mathbf{p_x^T} \\ \mathbf{p_y^T} \\ \mathbf{p_z^T} \end{bmatrix} \tag{6.9}$$

where I is the $[3, 3]$ identity matrix and $\mathbf{p_x^T}, \mathbf{p_y^T}, \mathbf{p_z^T}$, represent the projection operators in the x,y,z directions respectively.

6.4 Shallow Water Equations - Cartesian Form

As mentioned before, we are using the Shallow Water Equations (SWE) as a test case to evaluate the functioning of the framework of libraries that we have integrated. The 3-D Cartesian form of the full SWE on a rotating fluid is given by the following equations for time dependant velocity field $\mathbf{u}$ and geopotential height $\mathbf{h}$.

$$\frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla)\mathbf{u} - f(\mathbf{x} \times \mathbf{u}) - g\nabla h, \tag{6.10}$$

$$\frac{\partial h}{\partial t} = -\nabla \cdot (h\mathbf{u}). \tag{6.11}$$

where f is the Coroilis force, $\nabla = \partial_x \hat{i} + \partial_y \hat{j} + \partial_z \hat{k}$, $\mathbf{u} = u\hat{i} + v\hat{j} + w\hat{k}$ being the velocity vector, $\mathbf{x} = x, y, z^T$ represents the position vector, the geopotential height denoted by h and t denoting the time.

6.5 Gradient Operators

As introduced in the section 5.3 , we need gradient operators which when applied to the PDEs calculate the gradient of the equations.

Since we need all the formulations on the surface of the sphere, we need the gradients to be projected onto the surface of the sphere as well. The projection operator that we learnt about in the previous section will be applied to the gradient operators. ∇ is the divergence operator which, when projected on a tangential plane to the sphere, becomes the surface divergence operator for a vector field.

$$P\nabla = \begin{bmatrix} \mathbf{p_x} \cdot \nabla \\ \mathbf{p_y} \cdot \nabla \\ \mathbf{p_z} \cdot \nabla \end{bmatrix} \tag{6.12}$$

We replace all the ∇ operators in equation (6.10) with $P\nabla$. For our equations to be truly projected onto the sphere, we need to project the entire Right Hand Side (RHS) in (6.10) using the individual directional projection operators.

$$\frac{\partial u}{\partial t} = -\mathbf{p_x} \cdot \begin{bmatrix} (u \cdot \mathbf{P}\nabla)u + f(\mathbf{x} \times \mathbf{u})\hat{i} + g(\mathbf{p_x} \cdot \nabla)h \\ (u \cdot \mathbf{P}\nabla)v + f(\mathbf{x} \times \mathbf{u})\hat{j} + g(\mathbf{p_y} \cdot \nabla)h \\ (u \cdot \mathbf{P}\nabla)w + f(\mathbf{x} \times \mathbf{u})\hat{k} + g(\mathbf{p_z} \cdot \nabla)h \end{bmatrix} = -p_x \cdot RHS \tag{6.13}$$

$$\frac{\partial v}{\partial t} = -\mathbf{p_y} \cdot \text{RHS}$$

$$\frac{\partial w}{\partial t} = -\mathbf{p_z} \cdot \text{RHS}$$

$$\frac{\partial h}{\partial t} = -(\mathbf{P}\nabla) \cdot (h\mathbf{u})$$

Up until now, we have just formulated the equations in the 3D Cartesian format. But, we need the discretized form for solving these equations. This brings us to our next section.

6.6 Discretized SWE using RBF

According to the main theme of the project, we will be using the mesh-free method-RBF to descretize the SWE equations. Bringing together the information gathered in the previous sections, we can now write RBF discretized form of the SWE equation.

But first, let us formulate the RBF differentiation matrix, which will be integral to writing the discretized form of the SWE equation.

Given an atmospheric variable whose value is known at all node locations $\{x_j\}_{j=1}^{N}$ in the subdomain, we can write it as a linear expansion of the compactly supported RBF.

$$f(\mathbf{x}) = \sum_{j=1}^{N} c_j \varphi_j(r(\mathbf{x})) \tag{6.14}$$

The coefficients c_j will be determined by collocation. We will project this form onto the surface of the sphere for our problem statement. We will start with one direction x, and expand the definition in a similar fashion for y, z directions.

The following equation would be evaluated at each node $\{x_j\}_{j=1}^{N}$. Here, $\varphi_j(r(x))$ is a Wu-Wendland function centered at j. Based on the radius provided, it has $k = 1, \ldots n_j^R$ number of points in its neighborhood forming the stencil at the node j.

$$[p_x \cdot \nabla f(\mathbf{x})]\Big|_{\mathbf{x}=\mathbf{x_j}} = \sum_{k=1}^{n_j^R} c_j [\mathbf{p_x} \cdot \nabla \varphi_j(r(\mathbf{x}))]\Big|_{x=\mathbf{x_j}}, \quad j = 1, \ldots, N \tag{6.15}$$

$$\begin{aligned}
&= \sum_{k=1}^{N} c_j \left(x_j \mathbf{x_j^T x_k} - x_k \right) \frac{\varphi_k'(r(\mathbf{x_j}))}{r(\mathbf{x_j})} \quad j = 1, \ldots, N \\
&= B^x c \\
&= (B^x A^{-1}) f \\
&= D_R^x f
\end{aligned}$$

Here, A is the interpolation matrix introduced in the section 6.2 and the linear constant is given by $c = A^{-1} f$. D_R^x is the discrete RBF approximation of the projected gradient operator in x direction. Similarly we can expand this definition for $\{y, z\}$ directions.

$$[\mathbf{p_y} \cdot \nabla f(\mathbf{x})]\Big|_{x=x_j} = (B^y A^{-1}) f = D_R^y f \tag{6.16}$$

$$[\mathbf{p_y} \cdot \nabla f(\mathbf{x})]\Big|_{x=x_j} = (B^y A^{-1}) f = D_R^y f \tag{6.17}$$

We get a scalar value for each node. We can understand this better by looking at the shapes of the matrices obtained on the RHS of the equation.

$B_j^x : [1, n_j^R]$ $\quad A^{-1} : [n_j^R, n_j^R]$ $\quad f : [n_j^R, 1]$ $\quad D_R^x : [1, n_j^R]$

Using the formulation of projected gradient operator, we can formulate the descretized SWE described in the equation (6.13)

6.7 Right Hand Side formulation

In reference to the equation described in (6.13), we have four atmospheric variables. Three velocity field components: $\{u, v, w\}$ and one variable representing the geopotential height h. These are expressed in time dependant partial derivative functions of which the important component is the Right Hand Side (RHS) matrix. We can define the RHS matrix in the following way:

$$RHS_D = \begin{bmatrix} u \circ D_R^x \underline{u} + v \circ D_R^y \underline{u} + w \circ D_R^z \underline{u} \\ u \circ D_R^x \underline{v} + v \circ D_R^y \underline{v} + w \circ D_R^z \underline{v} \\ u \circ D_R^x \underline{w} + v \circ D_R^y \underline{w} + w \circ D_R^z \underline{w} \end{bmatrix} + f \begin{bmatrix} y \circ w - z \circ v \\ z \circ u - x \circ w \\ x \circ v - y \circ u \end{bmatrix} + g \begin{bmatrix} D_R^x \underline{h} \\ D_R^y \underline{h} \\ D_R^z \underline{h} \end{bmatrix} \tag{6.18}$$

Here, the $\{\underline{u}, \underline{v}, \underline{w}, \underline{h}\}$ are vectors of length n_j^R which contain the respective velocity field and height values at all the nodes inside the neighborhood cluster. The $\{u, v, w, h\}$ are the singular scalar velocity components and height values at the node (j) where we are calculating. The $\circ$ represents the element by element multiplication. As explained before, f is the Coriolis force computed for each node, g is the gravitational constant and h is the geopotential height.

$$\frac{\partial u}{\partial t} = -\mathbf{p_x} RHS_D \tag{6.19}$$

$$\frac{\partial v}{\partial t} = -\mathbf{p_y} RHS_D \tag{6.20}$$

$$\frac{\partial w}{\partial t} = -\mathbf{p_z} RHS_D \tag{6.21}$$

$$\frac{\partial h}{\partial t} = u \circ D_R^x \underline{h} + v \circ D_R^y \underline{h} + w \circ D_R^z \underline{h} + h \circ (D_R^x \underline{u} + D_R^y \underline{v} + D_R^z \underline{w}) \tag{6.22}$$

The above equations will be solved using a Runge-Kutta 4th (RK4) order time stepping loop.But before, we need to initialize the equations with appropriate initial value. 4

6.8 Initialization

Since, our problem statement defines the problem on the unit sphere, we need to scale some constants that are used in the equations to initialize the PDEs with the appropriate initial value.

6.8.1 Gravitational Constant

Newton's gravitational acceleration formula, $g = \frac{GM}{r^2}$,can be used as a reference for calculating the new g for our problem statement. Keeping the density same as the Earth's density, we can calculate the mass of a unit sphere. Now the known quantities are:

$$G = 6.673 \times 10^{-11}[M^{-1}L^3T^{-2}]$$
$$\rho = 5515[M^1L^{-3}]$$
$$r = 1[L^1]$$

The mass of the unit sphere is the product of density and volume of unit sphere : $M = 5515 \times 4.18879 = 2.3101 \times 10^4[M^1]$
By plugging this mass value in the gravitational acceleration formula, we get.

$$g = \frac{GM}{r^2} \tag{6.23}$$
$$= \frac{6.673 \times 10^{-11} \times 2.3101 \times 10^4}{1^2} \tag{6.24}$$
$$= 1.5415 \times 10^{-6}[L^1T^{-2}] \tag{6.25}$$

6.8.2 Geopotential height of Shallow Water Equations

The radius of earth is 6371000 m. Therefore using the most basic unitary method, we can calculate what would be the equivalent of 5000 m for a unit sphere. 5000 metres is an approximate height chosen for the shallow water testing condition.

$$h = \frac{5000}{6371000} = 7.848 \times 10^{-4}[L^1]$$

6.8.3 Velocity Calculation

Now, shallow water velocity calculation is important to determine the time step satisfying the Courant-Friedrich-Lewy (CFL) condition. The CFL condition states that the time step must be less than a certain time in explicit time-marching computer simulations, otherwise the simulation produces incorrect results. The CFL number is generally stipulated to be ≤ 1 and it is the ratio of the time step chosen, to the maximum time step possible.

In context of the time stepping simulation of a shallow water wave, the time step should be less than the time taken by the wave to reach from the node center to the end of the neighborhood cluster which is the stencil of influencing nodes.

The shallow water wave velocity is given by the following equation.

$$v_{\text{shallow}} = \sqrt{g \times h} \tag{6.26}$$

$$= \sqrt{1.5415 \times 10^{-6} \times 7.84810^{-4}} \tag{6.27}$$

$$= 3.4825 \times 10^{-5}[L^1 T^{-1}] \tag{6.28}$$

Let us assume Δt is the time step that we need to provide as an input for the RK4 time stepping algorithm. Following the definition of CFL condition, we can write the formulation as :

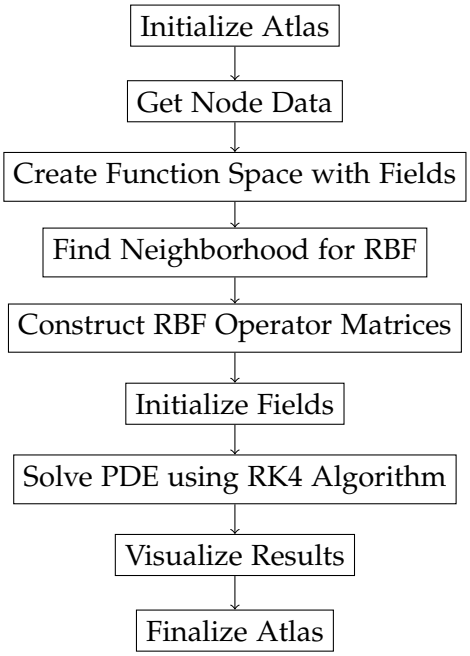
$$\Delta t = \frac{\text{Radial Distance inside neighborhood}}{v_{\text{shallow}}} \tag{6.29}$$

We shall use this equation for calculating our time step during the experiments.

Chapter 7

Detailed implementation discussion

So far we have studied the mathematical formulations of the different constituents of our project. In this chapter we will connect the information gathered together to arrive at our Python implementation of our project. We will discuss all the steps mentioned in the following diagram in detail along with some code snippets.



Each of these steps requires the installation of external Python supported libraries. The details are deferred to the Appendix section of this report.

7.1 Parallelization by Atlas

Recollecting the definition, 'Atlas is an ECMWF library for parallel data-structures supporting unstructured grids and function spaces, with the aim to investigate alternative more scalable dynamical core options for Earth System models, and to support modern interpolation and product generation software' [10]

This is a software developed by scientists from ECMWF. Since, this is an external software, we need it installed at the (local/remote) system of implementation, before we can use it in our program. We will not go into the steps here rather we would like to divert the reader to the Appendix A to go through the steps.

Atlas is an open source library written in C and Fortran programming languages, providing grids, mesh generation, and parallel data structures targeting Numerical Weather Prediction or Climate Model developments. Atlas4py is the Python bindings supporting the usage of the Atlas functionalities in Python based implementation.[13]

As highlighted time and again in this report, that the uniqueness of the approach lies in parallelizing the implementation from the start to end. Therefore, the entire code is wrapped inside the parallelizing feature of Atlas.[9]

Listing 7.1: Atlas initialize() and finalize()

```
atlas.initialize()
.
.
atlas.finalize()
```

7.1.1 Get coordinate information

The first step is to get the coordinate information. As explained in section 5.1 , we can read in longitude and latitude information from a `.nc` file. In our case, the file `grid.nc` has 20,480 nodes semi-uniformly distributed on a spherical geometry. Each node's location is defined in spherical coordinate system. We can derive the longitude and latitude information of all nodes in radians. The code snippet below shows the syntax for reading the NetCDF file.

Listing 7.2: Reading from NetCDF

```
f = nc.Dataset('/..Location../grid.nc')
print(f.variables.keys())

clon = f.variables['clon']
```

```
clat = f.variables['clat']
lon = clon[:]
lat = clat[:]
```

`f` is an object that stores the entire dataset available in the `grid.nc` file. By printing the `f.variables.keys()` we can get the names of each variable in the data set. Subsequently, individual variable objects can be accessed using `clon = f.variables['clon']` and stored into a local variable as illustrated in `lon = clon[:]`.

It is critical to know the measurement units (radians or degrees) of the information obtained from the grid file. And we will understand this criticality of this need in the upcoming subsections.

7.1.2 Creating `PointCloud` with Atlas features

The `PointCloud` feature of Atlas was extended to support the 3D unstructured nature of this project. The new functionality had the following key features which instrumental in this project:

- Unstructured grid: Atlas creates an unstructured grid using the longitude and latitude (lon-lat) information of the nodes. Now, we have received this information in the previous step. But, the catch is that Atlas needs this information in degrees. Therefore, it is critical to convert the `lon-lat` data from radians into degrees before passing it to Atlas.

  ```
  grid = atlas.UnstructuredGrid(lonlat[:, 0],
  lonlat[:, 1])
  ```

  Now, `grid` stores the unstructured grid that will be used to create the functionspace.

- Functionspace: Atlas creates a decomposed functionspace by taking the inputs: { `grid` , halo region radius, geometry}. Here, the halo region is specified as twice the radius used in our RBF functions.

  Listing 7.3: Creating functionspace

  ```
  functionspace = atlas.functionspace.
  PointCloud(grid, halo_radius=rbfradius * 2,
  geometry="UnitSphere")
  ```

- Creating and Accessing Fields: A very useful feature of the `functionspace` is the provision to create and access fields as Python numpy arrays. These are linked with the `functionspace` and other operations can be performed on these fields such as Halo Exchange.

  Here, `myfield` is a new field object created using `create_field()` functionality. It has 2 dimensions: first dimension - size of the functionspace

(rows) and second dimension - number of variable (columns). We can choose the number of variables in the argument and default is set to 1.

`make_view` functionality enables us to view, manipulate and use the fields as numpy arrays.

Listing 7.4: Creating functionspace

```
lonlat = atlas.make_view(functionspace.lonlat
)
myfield = functionspace.create_field(name="
swe_variables", variables=5, dtype = np.
float64)
uvwh = atlas.make_view(myfield)
```

`uvwh` is the field created to store the four atmospheric variables and coriolis function used in the shallow water equation test cases, namely the three velocity field components, geopotential height and the coriolis function ofcourse.

These decomposed subdomains have the following common fields which describe the function space. They can be accessed as numpy arrays using the template shown above.

- `lonlat`: longitudes and latitudes in degrees
- `size`: size of the subdomain which is the number of nodes present in one subdomain.
- `nb_parts`: number of subdomains in the decomposition
- `part`: rank of the subdomain
- `ghost`: an index for each node which indicates whether the node is an internal point (`ghost:0`) or a node belonging to the halo region (`ghost:1`)
- `partition`: partition owning nodes in the subdomain
- `remote_index`: provides the local index of the partition owning nodes
- `global_index`: provides global index for all nodes across all partitions

7.1.3 Find Neighborhood for RBF

Now, we have all the necessary ingredients to find the list of neighboring nodes for each node. This cluster of the nodes forms the RBF stencil for each node. Given a specific radius, Atlas finds the closest neighbors of each node

and returns their Global and Local indices for each subdomain. It employs kd-tree algorithm to find the list of closest nodes. In this implementation, a `Search` class is initialized with the functionspace. Then, `Search` class method `nearest_indices_within_radius()` takes the node index and radius as inputs and finds the nearest nodes within the provided radius. The code snippet below shows the class definition:

Listing 7.5: Search Functionality

```
class Search:
def __init__(self, functionspace):
    self.functionspace = functionspace
    self.lonlat = atlas.make_view(self.
    functionspace.lonlat)
    self.kdtree = atlas.IndexKDTree(geometry="
    UnitSphere")
    self.kdtree.build(self.lonlat)

def nearest_indices_within_radius(self, i, radius
):
    closest_indices = self.kdtree.
    closest_indices_within_radius(lon=self.lonlat
    [i, 0], lat=self.lonlat[i, 1],radius=radius)
    return closest_indices[1:]
```

The list is obtained for each node and stored for later use.

7.1.4 Construct RBF Operator Matrices

To arrive at the construction of RBF gradient operator (`D`) matrix, we need to construct interpolation (`A`) matrices for each node. This requires a loop which iterates over all the nodes, follows the necessary construction steps and collocates the gradient operators (`allDx,allDy,allDz`) corresponding to each node.

The following algorithm shows the steps executed inside the loop. Let us refer to this loop as the initialization loop. It only computes for the internal points of the subdomain.

Input: size of functionspace (`n_p`) , rbfradius, `ghost`
Output: Collocated gradient operators (`allD`) for all nodes.

The function that constructs the gradient operators is shown in listing 7.14. It follows the mathematical formulation described in (6.15):

Computed gradient operators for each node are collocated together and later used in the time stepping loop. (`allD`)

Algorithm 2 Initialization Loop

1: **for** `id` in `n_p` **do**
2: **if** `ghost` == 0 **then**
3: Calculate Nearest Neighbors
4: Construct A matrix using nearest neighbors and Wu-Wendland functions.
5: Invert the matrix A
6: Construct D matrix using inverse of A and the Cartesian coordinates at the node
7: **end if**
8: **end for**

7.1.5 Initialize Fields

Now that we have our gradient operators ready. The fields created in the listing 7.4 are initialized with test case values. The details of the test cases results are discussed in the chapter 8.

7.1.6 Solve PDEs using RK4 algorithm

We have already looked at the RK4 algorithm in section 5.5. Additionally, this subsection gives the overview of the time stepping algorithm implemented in this project. The following algorithm shows the major steps inside the loop. Let us refer to this loop as the time loop.

Input: timesteps (`n_timesteps`), timestep size (`dt`), initial conditions (`uvwh`), collocated gradient operators (`allD`))
Output: Field values at the next time step

Algorithm 3 Time Loop

1: **for** `i` in `n_timesteps` **do**
2: Halo Exchange of fields - Synchronization across subdomains
3: Calculate RK4 values for all fields
4: Update next time step values in uvwh using RK4 values
5: Update the iterator to next time step
6: **end for**

Line 2 in Algorithm 3, mentions Halo exchange of the fields. This functionality is provided by Atlas as part of the features of the `functionspace` It is a feature to synchronize the values of the fields with other parallel processes. Following code snippet shows the syntax of performing a halo exchange for the fields created in the `functionspace`

Listing 7.6: Halo Exchange

```
myfield.halo_dirty = True  # if set to False,
following halo_exchange will have no effect. Note it
was already True upon create_field
myfield.halo_exchange()
```

Once, we have the results from the time loop, we can visualize the results at periodic time steps. The following heading covers how visualization was achieved.

7.1.7 Visualize Results

To get nice global plots, Atlas provides another useful feature. It provides a gathering functionality to bring together the field data across all processes.

Listing 7.7: Gather

```
fs = myfield.functionspace
field_global = fs.create_field_global(name="
swe_variables_global", variables=5,dtype=np.float64)
functionspace.gather(myfield, field_global)
uvwh_global = atlas.make_view(field_global)
```

The global fields obtained, are then used for visualizing the results.

Chapter 8

Experiments and Results

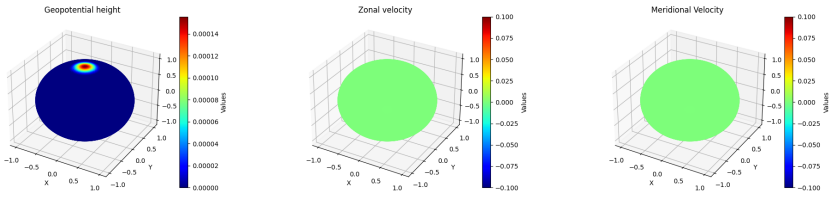
In this chapter, the results of the three test cases are showcased. Specifically, we shall work with the test cases number 0, 2 and 6 from the standard test cases set provided by David J. Williamson et al. for Numerical Approximations of SWE on a spherical geometry.[30].

The detailed definitions of each of these test cases are discussed in the chapter 4. A time step of 1200 seconds (20 minutes) was chosen to satisfy the CFL condition for shallow water waves. The results are presented after 1hr of time evolution with this time step for all the three test cases.

8.1 Test Case 0

Arctic pudding initially at rest

The initial condition of the velocity and height fields can be visualized in the figure 8.1. The plots are as per our expectation. We can see that zonal and meridional velocity fields are zero initially, which means the system is at rest. A small cosine bell can be seen in the plot for height field.

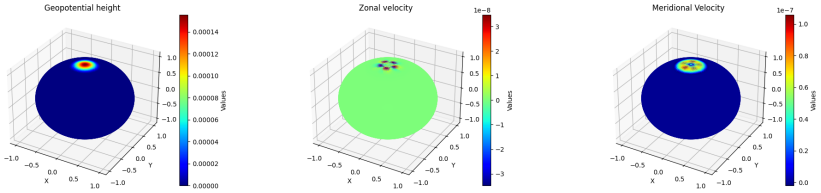


(a) Geopotential height **(b)** Zonal velocity **(c)** Meridional velocity

Figure 8.1: Test case 0: Initial conditions

These zonal and meridional velocities are converted into Cartesian velocity

components before starting the simulation. The shallow water flow introduced as a disturbance at the pole is expected to travel from pole to pole in the absence of the Coriolis force. But the simulation for test case 0 is not behaving as per the expectation.



(a) Geopotential height **(b)** Zonal velocity **(c)** Meridional velocity

Figure 8.2: Test case 0: After 1 hr, time step size of 1200 seconds

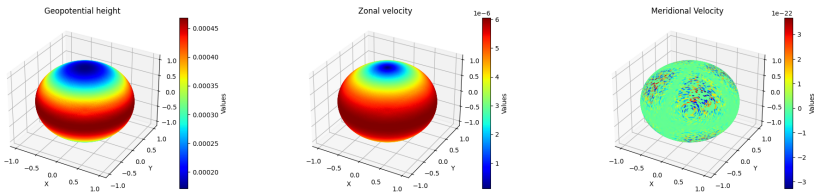
Interpretation of results :

- Meridional velocity: The plot shows some activity near the pole where the cosine bell of mass is introduced, but the direction of the velocity is wrong. The values are positive, hence the meridional velocity component is trying to keep the shape of the height bell curve intact at the pole, instead of acting southwards and facilitating the diffusion.
- Zonal velocity: The zonal component is expected to be mostly zero through out the simulation and much smaller than meridional component. But the plot shows that the zonal velocity component is non zero and introduces numerical instabilities in a symmetrical pattern near the pole. The values are also very similar to meridional velocity values, which is against the expectation.
- Geopotential height: Because of the action of the meridional component, the height plot remains constant event after 1 hour.

8.2 Test Case 2

Global Steady State Nonlinear Zonal Geostrophic Flow

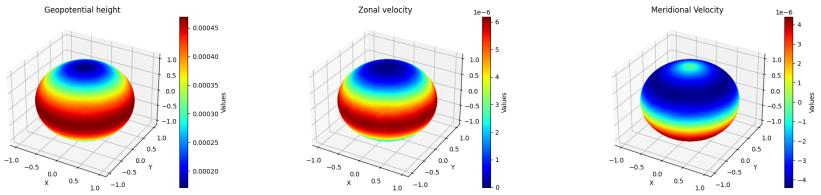
The initial condition of the velocity and height fields can be visualized in the figure 8.3. The plots are as per our expectation. We can see nice and smooth contours in the zonal velocity and height field. Meridional velocity plot is as per the expectation, which is to be completely zero, as we have not introduced any angular offset to the rotation of the sphere.



(a) Geopotential height **(b)** Zonal velocity **(c)** Meridional velocity

Figure 8.3: Test case 2: Initial conditions

These zonal and meridional velocities are converted into Cartesian velocity components, before starting the simulation. As time progresses, the sphere is slowly moving eastwards with the influence of the Coriolis force and the velocity and height fields are expected to deform slowly in the direction of the motion. But, yet again the plots after 1 hour are not showing expected results.



(a) Geopotential height **(b)** Zonal velocity **(c)** Meridional velocity

Figure 8.4: Test case 2: After 1 hr, time step size of 1200 seconds

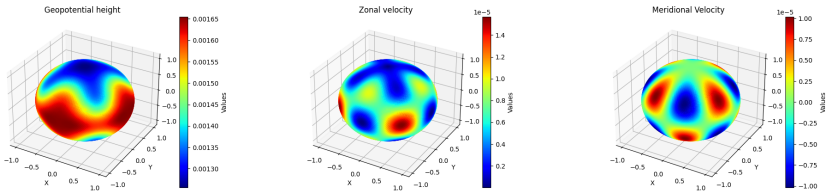
Interpretation of results:

- Meridional velocity: The entire sphere is divided into positive and negative values, with a zero value near the pole. This can be understood as the meridional velocity component trying to push the zonal and height fields towards the equatorial region from the north and the south.
- Zonal velocity: Naturally, we can see that there is some movement in the zonal velocity field, but it maintains its profile in the zonal direction.
- Geopotential height: The height field behaves similar to zonal velocity field, because of the influence of the meridional component.

8.3 Test Case 6

Rossby-Haurwitz Wave

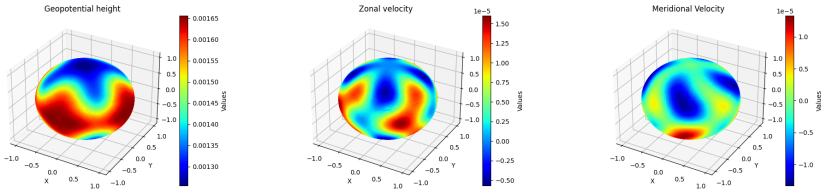
The initial condition of the velocity and height fields can be visualized in the figure 8.5. The plots are as per our expectation. We can see nice and smooth contours in the velocity and height fields. The four fingered pattern see in the height field plot is typical of this test case. It was introduced by setting the wave number $R = 4$ during the initialization.



(a) Geopotential height **(b)** Zonal velocity **(c)** Meridional velocity

Figure 8.5: Test case 6: Initial conditions

The simulation of Rossby-Haurwitz waves is by far the most complicated test case out of three test cases. It is expected to be quite stable in maintaining its profile. The sphere moves eastwards, similar to test case 2 and the velocity and height profiles are also expected to move along with the sphere by retaining their shape. Unfortunately, our simulation results just after 1 hour are not good results.



(a) Geopotential height **(b)** Zonal velocity **(c)** Meridional velocity

Figure 8.6: Test case 6: After 1 hr, time step size of 1200 seconds

Interpretation of results:

- Meridional and Zonal velocity: The velocity plots show deformations too soon in the simulation. This is an incorrect result. The zonal velocity field appears to be moving westwards and the meridional velocity field in the eastward direction.

- Geopotential height: The height plot remains quite stable after 1 hour of simulation. It is possible that this behaviour is due to the opposing effects of the velocity fields.

Evidently, the tests were unsuccessful in generating good results. The plots show multiple numerical instabilities in the result. At this stage, the mesh-free operators stand not validated. This could be because of a few open points:

1. There could be a bug in the program, which could be the reason for unexpected values. The bug could be because of internal memory allocation challenges presented by Python. Since, our expected range for gradient operators is in the order of 10^{-12} and 10^{-26} for zero values, one of the placeholders could be handling it poorly.
2. The testing methodologies were originally intended for the actual size of earth. Our problem statement however is defined for a unit sphere. There could be some unknown factor, which was not scaled down properly to the unit sphere.
3. The mathematical formulation developed for the current use case could use some modification for better results.The projection operators play a massive role in determining the accuracy of the results. The usage and application of the projection operators could be investigated further.

Lastly, computing the gradient operators for just one node could be a starting point for debugging the aforementioned open points. A simple script `onenode.py` is included in the github repository.[11] which calculates the gradient operators at one node in reference to the test case 0. This node is selected in such a way that we can be sure about the expected gradient operator values.

Chapter 9

Conclusions

This chapter concludes the research performed as part of this master thesis project. Let us recap the main objectives:

- To develop a completely parallel implementation in Python by integrating powerful libraries built for efficient performance on a supercomputer.
  - A completely parallel Python implementation was successfully developed. The program was tested with multiple nodes. Atlas was used for parallelization and it worked seamlessly.
- To use a mesh-free method, compactly supported Radial Basis Function, as a discretization technique for the formulation of operator approximations.
  - The new formulations for compactly supported RBF were introduced in this implementation. Due to the unexpected results in the test cases we can say that the Python implementation is not useful at this stage. Further investigation and study is required to arrive at successful results.
- To test the capability of an Industrial standard library for its efficiency in working on high resolution unstructured grids on a 3D spherical geometry.
  - State of the art functionalities provided by Atlas for 3D unstructured grids were introduced and implemented in this project. Tests were conducted to check the functioning of the features and the results are successful for all features. This project marks as the beginning of many more implementations consisting of these features to come.

Chapter 10

Future prospects

This master's thesis research explores the use of compactly supported Radial Basis Functions (RBF) for the approximation of numerical operators for 2D and 3D PDEs. The problem statement is defined around Shallow Water Equations (SWE) and some standard test cases available as part of old literature.

- We can introduce optimization in the construction of interpolation matrix. Currently, small matrices are constructed at each node which are of the size of neighborhood cluster. But, as an advancement, interpolation matrices can be constructed at the process level. The new interpolation matrices will be sparse. The advantage of working with sparse matrices is that, we can exploit power sparse libraries to obtain results faster.
- Since, this problem statement was an ideal case scenario, point cloud of semi - uniformly distributed nodes was created for just 1 layer close to the surface of the sphere. In a more realistic scenario, if we want to replicate earth's atmosphere layers, the point cloud should be extended to multiple layers. Atlas provides this feature to create a multilayered point cloud in a spherical geometry for global simulations.
- SWEs are a very good starting point to evaluate any algorithm developed for the intention of simulating atmospheric dynamics in NWP. Once the algorithm and the framework works for the SWE in an expected manner, the logical next step is to change the problem statement, introducing new set of equations and developing the mesh-free operators for them. Firstly, hydrostatic primitive Euler equations can be implemented and then one can progress to 3D non-hydrostatic fully compressible Euler equations.
- Finally, there is always a scope for improvement in the overall implementation itself. No code is perfect and there is a considerable scope for improvement in the current Python implementation. The next steps

would be to minimise the bottlenecks as much as possible, use memory efficient routines thereby reducing the lead time of the simulation.

Considering the overall theme of this research, very limited research is available that explores the use of radial basis functions for global Numerical Weather Prediction (NWP), especially compactly supported RBF, which are used in this project. The future of NWP lies in using Artificial Intelligence (AI) for weather forecasting. Significant research has been published recently, where RBF Neural networks are used for weather forecast.[22] [26] [4]

10.1 RBF Neural Network

RBF connect the scattered nodes in a domain. This feature of RBF is very similar to the fundamental concept of artificial neural networks where signals are passed between neural nodes. This similarity is being explored by researchers to use RBF as an initiation function for a neural network. We get a linear combination of the RBFs as an output of the neural network. [5]

A neural network mimics the neural connections of a biological organism. Therefore, these systems can be trained. It can learn from decades of atmospheric data collected. This upcoming research field is yet to reach the accuracy obtained in NWP models, but shows promise for an exciting new set of methods for weather forecasting.

Appendix A

Appendix

A.1 Supporting libraries in Python

- `Atlas Installation` It is important to make sure all the requirements are satisfied in the python environment for everthing to compile. Most important requirement in this case is MPI

Listing A.1: Atlas Installation

```
$ python3 -m venv atlas4py-venv
$ pip install -v git+https://github.com/
wdeconinck/atlas4py.git@feature/pointcloud
$ pip install numpy
```

- `pip install netCDF4` : Provides an object-oriented python interface to the netCDF version 4 library
- `pip install scipy` : Provides python supported linear algebraic routines
- `pip install matplotlib`: A popular open source library which provides plotting options for python

A.2 Acronyms

1. SWE - Shallow Water Equations
2. RBF - Radial Basis Functions
3. ICON - Icosahedral Non Hydrostatic
4. NWP - Numerical Weather Prediction
5. ECMWF - European Center for Mid-Range Weather Forecasts

6. SWES - Shallow Water Equations on Sphere
7. CFL - Courant-Friedrichs-Lewy
8. RBF-FD - Radial Basis Function generated Finite Differences
9. CSCS - Swiss National Supercomputing Centre

A.3 Complete python implementation

File name	Description
A_matrices.py	Constructs interpolation matrix and finds its inverse
atlas_func_interface.py	Contains the neighborhood search functionality of Atlas
atlas_main.py	The main file which drives the implementation
construct_rhsd.py	Constructs the right hand side for partial derivates of the fields
helperfuncs.py	Contains all the small helper functions
initializefields.py	Initializes data according to test case
operator_matrices.py	Constructs the gradient operator
read_netcdf_file.py	Reads longitude and latitude data from a netcdf file
testcases.py	Contains the initializing functions for test cases: 0,2,6
visualization.py	Contains the plotting function for height, meridional and zonal fields
WuWendland_functions.py	Contains Wu-Wendland functions and their derivative functions

Table A.1: Python scripts description

The python code for the implementation of the problem statement was completely developed by the author for this project. Entire programming is done in python. This python code is also available as a public GitHub repository. [11]

This section simply provides the python codes developed for the parallel implementation.The listings refer to individual python scripts titled same as the caption. The table A.1 describes the contents of each python script and they can be found listed in alphabetical order.

Listing A.2: A_matrices

```
############
# INPUT : An 2D array of coordinates which form the
neighborhood cluster for the node at which we are
calculating
# OUTPUT : Returns A (interpolation matrix) and invA
(inverse of interpolation matrix)
# WORKING : Can change function call statement in
line #27 and line #37 for a different Wendland
function. Check the file WuWendland_functions.py

#############

import numpy as np
import scipy.linalg as scp
import numpy.linalg as linalg
from WuWendland_functions import *
from helperfuncs import *


def constructA(xyz_r):

        n = len(xyz_r) #gets the size of the matrix
        A = np.zeros((n, n))  #declares a matrix with
         zeros for the construction of our
        interpolant matrix A


        # diagonal elements
        for i in range(n):
            A[i][i] = wendland1(0) #can change as per
             need

        #elements below and above diagonal
        for i in range(n):
            for k in range(n):
                if k < i :
                    r = eucl_norm(xyz_r[i],xyz_r[k])
                    #sends two tuples and gets the
                    norm back
                    r=r/0.065 #scaling to match the
                    Wendland functions
                    A[i][k] = wendland1(r) #can
                    change as per need
                    A[k][i] = A[i][k]
```

```
        invA = inverta(A)

        return A, invA


def inverta(A):

    pl,u = scp.lu(A, permute_l = True)
    u_inv = linalg.inv(u)
    pl_inv = linalg.inv(pl)

    inv_a = np.matmul(u_inv,pl_inv)

    return inv_a
```

Listing A.3: atlas_func_interface

```
#############
# INPUT : Index of the node j where we are
calculating, radius for the RBF stencil
# OUTPUT : List of indices of nodes which are closest
 to the node j within the provided radius
##############

import atlas4py as atlas
import math
import numpy as np


class Search:
    def __init__(self, functionspace):
        self.functionspace = functionspace
        self.lonlat = atlas.make_view(self.
        functionspace.lonlat)
        self.kdtree = atlas.IndexKDTree(geometry="
        UnitSphere")
        self.kdtree.build(self.lonlat)

    def nearest_indices_within_radius(self, i, radius
    ):
        # radius is in metres on Earth geometry
        closest_indices = self.kdtree.
        closest_indices_within_radius(lon=self.lonlat
```

```
        [i, 0], lat=self.lonlat[i, 1], radius=radius)
        return closest_indices[1:]  # first point is
        "i" itself, so skip it
```

Listing A.4: atlas_main

```
# Master Thesis Project

# This is the main python script for the project
# Step by step , it will call and execute all the
functions.
#   1. Reads Coordinates list from Netcdf4 file
#   2. Provides coordinates input to Atlas Point
Cloud script
#   3. Gets Indices back from Atlas Script
#   4. Creates A Matrix and its inverse
#   5. Creates Differentiation Matrix D : B * A inv
#   6. Initializes fields with test case conditions
#   7. RK4 Time Stepping Loop
#   8. Run Test Cases
#   9. Publish Results


#!/usr/bin/env python3
import atlas4py as atlas
import numpy as np
import time

from operator_matrices import *
from atlas_func_interface import *
from read_netcdf_file import *
from A_matrices import *
from initializefields import *
from visualization import *
from rk4 import *
from construct_rhsd import *


atlas_start = time.time()
atlas.initialize() #initializes atlas and MPI

#Constants and Variables
myradius = 0.065 #can generalize later
nrj_size_list = []
allnearest = []
```

```
allDx = []
allDy = []
allDz = []


lonlat = read_data_netcdf() #Read data from a netcdf
file
grid = atlas.UnstructuredGrid(lonlat[:, 0], lonlat[:,
 1]) #Create Unstructured Grid

#Create functionspace + partitioning #levels in the
pointcloud implementation
functionspace = atlas.functionspace.PointCloud(grid,
halo_radius=myradius * 2, geometry="UnitSphere")

###Access fields as numpy arrays, if needed
lonlat = atlas.make_view(functionspace.lonlat) #
longitude latitude in degrees
ghost = atlas.make_view(functionspace.ghost) # ghost:
 1 for ghost, 0 for owned
partition = atlas.make_view(functionspace.partition)
#partition owning point (0-based)
remote_index = atlas.make_view(functionspace.
remote_index) # local index on partition owning point
 (careful, 1-based when atlas_HAVE_FORTRAN)
global_index = atlas.make_view(functionspace.
global_index) # global index across partitions (
always 1-based)



xyz = getcartesian(lonlat)
###Create Fields
#first dimension - size of the functionspace (rows)
#second dimension - number of variables (columns)
internal = [ i for i in range(functionspace.size) if
ghost[i] == 0 ]
#field = functionspace.create_field(name="mycoords",
variables=3,levels = 10, dtype = np.float64)

myfield = functionspace.create_field(name="
swe_variables", variables=5, dtype = np.float64)
uvwh = atlas.make_view(myfield)
```

```
mycoords = functionspace.create_field(name="lonlat",
variables=2, dtype = np.float64)
lonlat_sd = atlas.make_view(mycoords)

n_p = functionspace.size
search = Search(functionspace) #initializes the
Search class with functionspace

initloop_start = time.time()
###################  Loop to construct RBF Operator
matrice  ############################
initloop_start = time.time()
for index in range(n_p):  # n_p
    if ghost[index] == 0:

        nearest = search.
        nearest_indices_within_radius(index, myradius
        )
        allnearest = np.append(allnearest, nearest)
        nrj_size_list = np.append(nrj_size_list, len(
        nearest))
        xyz_r = getneighcoords(nearest, xyz) #Gets
        the list of neighborhood cartesian
        coordinates for further use

        #call function to create A, A inverse and
        save them
        A, invA = constructA(xyz_r)

        Xj = xyz[index]
        #call function to create gradient operators
        Dx, Dy, Dz = differentiation(invA,xyz_r,Xj)

        allDx = np.append(allDx, Dx)
        allDy = np.append(allDy, Dy)
        allDz = np.append(allDz, Dz)

initloop_end = time.time()
allD = np.column_stack([allDx, allDy, allDz])


#
#####################################################################
```

```
#function call to get initial conditions
fields = set_initial_conditions(xyz, n_p, ghost,
lonlat)

#fill values in the functionspace variables
for n in range(n_p):
  if not ghost[n]:
    lonlat_sd[n,0] = lonlat[n,0]
    lonlat_sd[n,1] = lonlat[n,1]

    uvwh[n,0] = fields[n,0]
    uvwh[n,1] = fields[n,1]
    uvwh[n,2] = fields[n,2]
    uvwh[n,3] = fields[n,3]
    uvwh[n,4] = fields[n,4]

  else:
     lonlat_sd[n,0] = lonlat[n,1] = 0.0
     uvwh[n,0] = 0.0
     uvwh[n,1] = 0.0
     uvwh[n,2] = 0.0
     uvwh[n,3] = 0.0
     uvwh[n,4] = 0.0


#Update values (Halo exchange)
myfield.halo_dirty = True
myfield.halo_exchange()

mycoords.halo_dirty = True
mycoords.halo_exchange()

timeloop_start = time.time()

n_timesteps = 1 #12 days (12 * 86400/1200)


#
############################################################

def get_rk4_values(uvwh, dt, nrj_size_list,
allnearest,xyz, allD, ghost):
```

```
#Here uvwh has the size of entire subdomain n_p
including Halo Region
num = len(uvwh)

rk_u = np.zeros((num,4))
rk_v = np.zeros((num,4))
rk_w = np.zeros((num,4))
rk_h = np.zeros((num,4))
uvwh0 = uvwh
num = len(uvwh)

#d1 = dt*rhs(k1)
rk_u[:,0], rk_v[:,0], rk_w[:,0], rk_h[:,0]  =
construct_rhsd(nrj_size_list, allnearest, uvwh,
xyz, allD, ghost)
rk_u[:, 0] = rk_u[:, 0] * dt
rk_v[:, 0] = rk_v[:, 0] * dt
rk_w[:, 0] = rk_w[:, 0] * dt
rk_h[:, 0] = rk_h[:, 0] * dt


#******************  k2  **************#

#k2 = uvwh0 + 0.5*d1
for i in range(num):
    uvwh[i, 0] = uvwh0[i, 0] + (rk_u[i,0]/2)
    uvwh[i, 1] = uvwh0[i, 1] + (rk_v[i,0]/2)
    uvwh[i, 2] = uvwh0[i, 2] + (rk_w[i,0]/2)
    uvwh[i, 3] = uvwh0[i, 3] + (rk_h[i,0]/2)


#Update values (Halo exchange)
myfield.halo_dirty = True
myfield.halo_exchange()

# d2 = dt*rhs(k2)
rk_u[:, 1], rk_v[:, 1], rk_w[:, 1], rk_h[:, 1] =
construct_rhsd(nrj_size_list, allnearest, uvwh,
xyz, allD, ghost)
rk_u[:, 1] = rk_u[:, 1] * dt
rk_v[:, 1] = rk_v[:, 1] * dt
rk_w[:, 1] = rk_w[:, 1] * dt
rk_h[:, 1] = rk_h[:, 1] * dt
```

```
#*************    k3   *************#

#k3 = uvwh0 +0.5*d2
for i in range(num):
    uvwh[i, 0] = uvwh0[i, 0] + (rk_u[i, 1]/2)
    uvwh[i, 1] = uvwh0[i, 1] + (rk_v[i, 1]/2)
    uvwh[i, 2] = uvwh0[i, 2] + (rk_w[i, 1]/2)
    uvwh[i, 3] = uvwh0[i, 3] + (rk_h[i, 1]/2)

#Update values (Halo exchange)
myfield.halo_dirty = True
myfield.halo_exchange()

 # d3 = dt*rhs(k3)
rk_u[:, 2], rk_v[:, 2], rk_w[:, 2], rk_h[:, 2] =
construct_rhsd(nrj_size_list, allnearest, uvwh,
xyz, allD, ghost)
rk_u[:, 2] = rk_u[:, 2] * dt
rk_v[:, 2] = rk_v[:, 2] * dt
rk_w[:, 2] = rk_w[:, 2] * dt
rk_h[:, 2] = rk_h[:, 2] * dt

# *************    k4   *************#

#k4 = uvwh0 + d3
for i in range(num):
    uvwh[i, 0] = uvwh0[i, 0] + (rk_u[i, 2])
    uvwh[i, 1] = uvwh0[i, 1] + (rk_v[i, 2])
    uvwh[i, 2] = uvwh0[i, 2] + (rk_w[i, 2])
    uvwh[i, 3] = uvwh0[i, 3] + (rk_h[i, 2])

#Update values (Halo exchange)
myfield.halo_dirty = True
myfield.halo_exchange()

# calculation of rhs with k4 as input
rk_u[:, 3], rk_v[:, 3], rk_w[:, 3], rk_h[:, 3] =
construct_rhsd(nrj_size_list, allnearest, uvwh,
xyz, allD, ghost)

# d4 = dt*rhs(k4)
rk_u[:, 3] = rk_u[:, 3] * dt
rk_v[:, 3] = rk_v[:, 3] * dt
rk_w[:, 3] = rk_w[:, 3] * dt
```

```
    rk_h[:, 3] = rk_h[:, 3] * dt

    return rk_u, rk_v, rk_w, rk_h


#
######################################################################

##### TIME STEPPING LOOP #######
#
######################################################################

for i in range(n_timesteps):

    rk_u = np.zeros((n_p,4))
    rk_v = np.zeros((n_p,4))
    rk_w = np.zeros((n_p,4))
    rk_h = np.zeros((n_p,4))

    dt = 1200 #20 mins

    uvwh0 = uvwh #store previous time step values

    rk_u, rk_v, rk_w, rk_h = get_rk4_values(uvwh, dt,
     nrj_size_list, allnearest,xyz, allD, ghost)

    for k in range(n_p):
        if not ghost[k]:

            #Calculate uvwh at next timestep (
            overwriting): need all RHS values
            uvwh[k,0] = uvwh0[k,0] + ((rk_u[k,0] +
            rk_u[k,3]) + 2*(rk_u[k,1]+rk_u[k,2]))/6
            uvwh[k,1] = uvwh0[k,1] + ((rk_v[k,0] +
            rk_v[k,3]) + 2*(rk_v[k,1]+rk_v[k,2]))/6
            uvwh[k,2] = uvwh0[k,2] + ((rk_w[k,0] +
            rk_w[k,3]) + 2*(rk_w[k,1]+rk_w[k,2]))/6
            uvwh[k,3] = uvwh0[k,3] + ((rk_h[k,0] +
            rk_h[k,3]) + 2*(rk_h[k,1]+rk_h[k,2]))/6

    #Update values after each time step (Halo
    exchange)
    myfield.halo_dirty = True
    myfield.halo_exchange()
```

```
timeloop_end = time.time()


#Update values (Halo exchange)
myfield.halo_dirty = True
myfield.halo_exchange()

mycoords.halo_dirty = True
mycoords.halo_exchange()


#create global variables and gather individual
process data to the global variable at process 0
fs = myfield.functionspace
field_global = fs.create_field_global(name="
swe_variables_global", variables=5,dtype=np.float64)
functionspace.gather(myfield, field_global)
uvwh_global = atlas.make_view(field_global)

mc = mycoords.functionspace
lonlat_global = mc.create_field_global(variables=2)
functionspace.gather(mycoords, lonlat_global)
lonlat_global_coords = atlas.make_view(lonlat_global)


if functionspace.part ==0:
    plot_global(uvwh_global,lonlat_global_coords)

print("Fin.")
atlas.finalize()
```

Listing A.5: construct_rhsd

```
import numpy as np
#from operator_matrices import *
import math
from helperfuncs import *
#from testcases import *


def construct_rhsd(nrj_size_list, allnearest, uvwh,
xyz, allD, ghost):
    n_p = len(uvwh)
    rhsd = np.zeros(3)
```

```
it = 0
termA = np.zeros(3)
termB = np.zeros(3)
termC = np.zeros(3)

Ru = np.zeros(n_p)
Rv = np.zeros(n_p)
Rw = np.zeros(n_p)
Rh = np.zeros(n_p)

g = 1.5454e-6
l = 0
f = uvwh[:,4]

# Calculating RHS values for each of the internal
 points
for i in range(n_p):
    if not ghost[i]:

        k = int(nrj_size_list[l]) #Check the
        requirement of int typecasting here
        l= l+1
        Dnx, Dny, Dnz, nearest =
        get_Dnxyz_from_allD(allD,allnearest, it,k
        )

        it = it + k

        xyz_r = getneighcoords(nearest, xyz)
        uvwh_r = get_uvwh_r(uvwh, nearest)

        u = uvwh_r[:, 0]
        v = uvwh_r[:, 1]
        w = uvwh_r[:, 2]
        h = uvwh_r[:, 3]

        termA[0] = (uvwh[i,0]*(np.dot(np.
        transpose(Dnx),u)) +
             uvwh[i,1]*(np.dot(np.transpose(Dny),
             u)) +
             uvwh[i,2]*(np.dot(np.transpose(Dnz),
             u)))

        termA[1] = (uvwh[i, 0] * (np.dot(np.
```

```
transpose(Dnx), v)) +
            uvwh[i, 1] * (np.dot(np.
            transpose(Dny), v)) +
            uvwh[i, 2] * (np.dot(np.
            transpose(Dnz), v)))

termA[2] = (uvwh[i, 0] * (np.dot(np.
transpose(Dnx), w)) +
        uvwh[i, 1] * (np.dot(np.transpose
        (Dny), w)) +
        uvwh[i, 2] * (np.dot(np.transpose
        (Dnz), w)))

#f. [3,1]
termB[0] = f[i]*((xyz[i,1]*uvwh[i,2]) - (
xyz[i,2]*uvwh[i,1]))
termB[1] = f[i]*((xyz[i,2]*uvwh[i,0]) - (
xyz[i,0]*uvwh[i,2]))
termB[2] = f[i]*((xyz[i,0]*uvwh[i,1]) - (
xyz[i,1]*uvwh[i,0]))

#g.[Dnx Dny Dnz]. h
termCx= np.transpose(Dnx)
termCy= np.transpose(Dny)
termCz= np.transpose(Dnz)

term_c = np.row_stack([termCx,termCy,
termCz])
termC = g*(np.dot(term_c,h))


rhsd[0] = termA[0] + (termB[0]) + (termC
[0])
rhsd[1] = termA[1] + (termB[1]) + (termC
[1])
rhsd[2] = termA[2] + (termB[2]) + (termC
[2])

#Get px, py, pz at the node where we are
calculating
px, py, pz = getpxyz(xyz[i,:])


Rh[i] = uvwh[i, 0]*(np.dot(np.transpose(
```

```
            Dnx),h)) + uvwh[i, 1]*(np.dot(np.
            transpose(Dny),h)) + uvwh[i, 2]*(np.dot(
            np.transpose(Dnz),h)) + uvwh[i, 3]*(np.
            dot(np.transpose(Dnx),u) + np.dot(np.
            transpose(Dny),v)+ np.dot(np.transpose(
            Dnz),w))

            Ru[i] = -np.dot(px, rhsd)
            Rv[i] = -np.dot(py, rhsd)
            Rw[i] = -np.dot(pz, rhsd)


    return Ru, Rv, Rw, Rh
```

Listing A.6: helperfuncs

```
import numpy as np
import math
#from initializefields import *
from testcases import *


def getpxyz(xyz):

    px = np.zeros(3)
    py = np.zeros(3)
    pz = np.zeros(3)

    x = xyz[0]
    y = xyz[1]
    z = xyz[2]

    px[0] = (1 - (x ** 2))
    px[1] = (0 - (x * y))
    px[2] = (0 - (x * z))

    py[0] = (0 - (x * y))
    py[1] = (1 - (y ** 2))
    py[2] = (0 - (y * z))

    pz[0] = (0 - (x * z))
    pz[1] = (0 - (y * z))
    pz[2] = (1 - (z ** 2))

    px = px[np.newaxis,:] #row matrix
```

```
    py = py[np.newaxis,:]
    pz = pz[np.newaxis,:]

    return px, py, pz

#need to get neighborhood Dnx, Dny, Dnz (saved
sequentially for each point)
def get_Dnxyz_from_allD(allD, allnearest, it, k):

    itx = it+k
    Dnx = allD[it: itx, 0]
    Dny = allD[it: itx, 1]
    Dnz = allD[it: itx, 2]
    nearest = allnearest[it:itx]

    return Dnx[:,np.newaxis], Dny[:,np.newaxis], Dnz
    [:,np.newaxis], nearest


def get_uvwh_r(uvwh,nearest):
    uvwh_r = np.zeros([len(nearest),4])
    for n in range(len(nearest)):
        uvwh_r[n,0] = uvwh[int(nearest[n]),0]
        uvwh_r[n,1] = uvwh[int(nearest[n]),1]
        uvwh_r[n,2] = uvwh[int(nearest[n]),2]
        uvwh_r[n,3] = uvwh[int(nearest[n]),3]

    return uvwh_r



def getcartesian(lonlat):

    ## Convert lon lat from degrees to radians.
    Uncomment if needed
    lon = np.radians(lonlat[:,0])
    lat = np.radians(lonlat[:,1])

    # R = 6371229 Radius of earth in meters
    R = 1.0  # radius of unit sphere
    x = R * np.cos(lat) * np.cos(lon)
    y = R * np.cos(lat) * np.sin(lon)
    z = R * np.sin(lat)
```

```
    xyz = np.column_stack([x,y,z])

    return xyz


def eucl_norm(x1,x2):
    return np.linalg.norm(x1-x2)

def getneighcoords(my_list, coords):
    xyz_r = np.zeros((len(my_list),3))
    #print("My list:", my_list)
    for i in range(len(my_list)):
        k = int(my_list[i])
        xyz_r[i,:] = coords[k]

    return xyz_r


def gvec2cvec_sphere(lon_fun, lat_fun, lam, th,ghost)
:
    l = len(lat_fun)
    uvw = np.zeros((l,3))

    for i in range(l):

            s2c_u = np.array([-np.sin(lam[i]), np.cos
            (lam[i]), 0.0])
            s2c_v = np.array([-np.cos(lam[i])*np.sin(
            th[i]), -np.sin(lam[i])*np.sin(th[i]), np
            .cos(th[i])])
            uvw[i,0] = s2c_u[0]*lat_fun[i] + s2c_v
            [0]*lon_fun[i]
            uvw[i,1] = s2c_u[1]*lat_fun[i] + s2c_v
            [1]*lon_fun[i]
            uvw[i,2] = s2c_u[2]*lat_fun[i] + s2c_v
            [2]*lon_fun[i]


    return uvw


def cvec2gvec_sphere(uvwh, lon, lat):

    l = len(lon)
```

```
    p_gu = np.zeros(l)
    p_gv = np.zeros(l)
    p_norm = np.zeros(l)

    for i in range(l):
        z_sln = np.sin(lon[i])
        z_cln = np.cos(lon[i])
        z_slt = np.sin(lat[i])
        z_clt = np.cos(lat[i])

        p_gu[i] = (z_cln*uvwh[i,1]) - (z_sln*uvwh[i
        ,0])
        tmp = (z_cln*uvwh[i,0]) + (z_sln*uvwh[i,1])
        tmp = (z_slt*tmp)
        p_gv[i] = (z_clt*uvwh[i,2]) - tmp

    return p_gu, p_gv

def validate_halo_exchange(uvwh,xyz, n_p,ghost):
    vt = 1
    for n in range(n_p):
        if ghost[n]:
            if (uvwh[n,3] != 4*(xyz[n,0]**2 +xyz[n
            ,1]**2 +xyz[n,2]**2)):
                print("Halo␣exchange␣failed")
                vt = 0

    return vt
```

Listing A.7: initializefields

```
import numpy as np
import math
from helperfuncs import *
from testcases import *


def set_initial_conditions(xyz, n_p, ghost,lonlat):

    N = n_p #internal+halo , 5 field columns
    f = np.zeros(N)

    fields = np.zeros((N,5))
    lam = np.radians(lonlat[:,0])
```

```
    th = np.radians(lonlat[:,1])

    x = xyz[:,0]
    y = xyz[:,1]
    z = xyz[:,2]

    v_lon = np.zeros(N)
    v_lat = np.zeros(N)
    h = np.zeros(N)

    for n in range(N):
        if not ghost[n]: #calculating all internal
        values
            v_lon[n], v_lat[n], h[n], f[n] =
            test0_fun(lam[n],th[n]) #can change to
            test2_fun() , test6_fun()

        else:
            v_lon[n] = v_lat[n] = h[n] = 0.0
            f[n] = 0.0

    uvw = gvec2cvec_sphere(v_lon, v_lat, lam, th,
    ghost)

    for n in range(n_p):
        if not ghost[n]:
            fields[n,0] = uvw[n,0]
            fields[n,1] = uvw[n,1]
            fields[n,2] = uvw[n,2]
            fields[n,3] = h[n]
            fields[n,4] = f[n]

        else:
            fields[n,0] = 0.
            fields[n,1] = 0.
            fields[n,2] = 0.
            fields[n,3] = 0.
            fields[n,4] = 0.


    return fields
```

Listing A.8: operator_matrices

```
import numpy as np
```

```
from A_matrices import *
from WuWendland_functions import *

def differentiation (inv_A,xyz_r,Xj):

    # B = [ xk.(Xj*.Xk) -xj ]*{phi'(r(Xj))/r(Xj)}
    # inv_A = inverse of A matrix calculated before
    # Xj = xyz coordinates where the neighborhood is
    calculated
    # xk are the list of coordinates in the
    neighborhood
    # xyz_r = the coordinate list of all neighborhood
     points
    # D = B * inv_A

    n = len(xyz_r)
    Bx = np.zeros(n)
    By = np.zeros(n)
    Bz = np.zeros(n)

    for i in range(n):

        r_x = eucl_norm(xyz_r[i,:],Xj)/0.065 #scaling
         to match wendland functions
        mult_constant = wendland1_prime(r_x) / r_x


        #[1,nrj] = ([1,1]*(np.matmul([1,3],[3,nrj]) -
         [1,nrj]) * [1,nrj]
        Bx[i] = ((xyz_r[i,0]*(np.dot(Xj,(xyz_r[i,:]))
        )) - Xj[0])*mult_constant
        By[i] = ((xyz_r[i,1]*(np.dot(Xj,(xyz_r[i,:]))
        )) - Xj[1])*mult_constant
        Bz[i] = ((xyz_r[i,2]*(np.dot(Xj,(xyz_r[i,:]))
        )) - Xj[2])*mult_constant

    #[1,nrj] = np.matmul( [1,nrj],[nrj,nrj] )
    Bx = Bx[np.newaxis,:]
    By = By[np.newaxis,:]
    Bz = Bz[np.newaxis,:]

    #Bxyz = np.row_stack([Bx,By,Bz])

    Dx = np.matmul(Bx,inv_A)
```

```
    Dy = np.matmul(By,inv_A)
    Dz = np.matmul(Bz,inv_A)

    #Dxyz = np.row_stack([Dx,Dy,Dz])

    return Dx, Dy, Dz
```

Listing A.9: read_netcdf_file

```
# Master Thesis Project
# This python script reads data from a given NetCDF
File
# Latitudes and Longitudes are in Radians.
# Lon Lat Coordinates are returned in degrees for use
 in Atlas

import netCDF4 as nc
import numpy as np
import math

def read_data_netcdf():

    f = nc.Dataset('/users/ldevulap/MyThesis2023/
    projectrbf/GRID_FILES/grid.nc')
    #print(f.variables.keys())

    clon = f.variables['clon']
    clat = f.variables['clat']
    lon = clon[:]
    lat = clat[:]

    ## Convert lon lat from radians to degres
    lon[:] = np.degrees(lon[:])
    lat[:] = np.degrees(lat[:])


    # R = 6371229 Radius of earth in meters
    R = 1 # radius of unit sphere
    x = R * np.cos(lat) * np.cos(lon)
    y = R * np.cos(lat) * np.sin(lon)
    z = R * np.sin(lat)

    lonlat = np.column_stack([lon,lat])

    coordxyz = np.column_stack([x,y,z])
```

```
    return lonlat
```

Listing A.10: testcases

```
import math
import numpy as np



def test2_fun(lam,th):
    g = 1.5454e-6;
    radius = 1.0
    uu0 = 2 * np.pi * radius / (12 * 86400);
    angle = 0.0 #0.349
    h0 = 2.94e4 / (g*6371000*6371000); #1.9024e-4
    Omega = 7.292e-5;

    coriolis_fun = lambda lam, th: 2 * Omega * (np.
    sin(th) * np.cos(angle) - np.cos(th) * np.cos(lam
    ) * np.sin(angle))

    h0_fun = lambda lam, th: h0 - (1 / g) * (radius *
     Omega * uu0 + uu0 ** 2 / 2) * (np.sin(th) * np.
    cos(angle) - np.cos(lam) * np.cos(th) * np.sin(
    angle)) ** 2

    lat_fun = uu0 * (np.cos(th) * np.cos(angle) + np.
    sin(th) * np.cos(lam) * np.sin(angle))
    lon_fun = -uu0 * np.sin(angle) * np.sin(lam)

    h = h0_fun(lam,th)

    f = coriolis_fun(lam,th)

    return lon_fun,lat_fun, h, f


def test6_fun(lam,th):

    g = 1.5454e-6;
    Omega = 7.292e-5;
    omega = 7.848e-6;
    K = omega;
    h0 = 1.25568e-3;#8000/6371000
```

```
    R = 4
    radius = 1.0
    uu0 = 2 * np.pi * radius / (12 * 86400);
    angle = 0.0

    coriolis_fun = lambda lam, th: 2 * Omega * (np.
    sin(th))
    #h0 = 2.94e4 / (g*6371000*6371000); #1.9024e-4

    A_fun = lambda lam, th: 0.5 * omega * (2 * Omega
    + omega) * np.cos(th) ** 2 + 0.25 * K * K * np.
    cos(th) ** (2 * R) * ((R + 1) * np.cos(th) ** 2 +
     (2 * R * R - R - 2) - 2 * R * R * np.cos(th) **
    (-2))
    B_fun = lambda lam, th: 2 * (Omega + omega) * K *
     np.cos(th) ** R * ((R * R + 2 * R + 2) - (R + 1)
     ** 2 * np.cos(th) ** 2) / ((R + 1) * (R + 2))
    C_fun = lambda lam, th: 0.25 * K * K * np.cos(th)
     ** (2 * R) * ((R + 1) * np.cos(th) ** 2 - (R +
    2))

    h0_fun = lambda lam, th: h0 + radius * (radius /
    g )* (A_fun(lam, th) + B_fun(lam, th) * np.cos(R
    * lam) + C_fun(lam, th) * np.cos(2 * R * lam))
    u_fun = lambda lam, th: radius * omega * np.cos(
    th) + radius * K * np.cos(th) ** (R - 1) * (R *
    np.sin(th) ** 2 - np.cos(th) ** 2) * (np.cos(R *
    lam))
    v_fun = lambda lam, th: (-radius * K * R * np.cos
    (th) ** (R - 1) * np.sin(th) * np.sin(R * lam))


    lat_fun = u_fun(lam,th)
    lon_fun = v_fun(lam,th)
    h = h0_fun(lam,th)
    f = coriolis_fun(lam,th)

    return lon_fun,lat_fun, h, f


def test0_fun(lam,th):

    h0 = 1000/6371000
    h1 = 500/6371000
```

```
    a = 1.0 #radius of unit sphere

    R = a/3

    snp = np.sin(np.pi/2)
    csp = np.cos(np.pi/2)

    r = a*np.arccos(snp*np.sin(th) + csp*np.cos(th)*
    np.cos(lam-(np.pi)))

    if r<R:
        h = (h0/2)*(1 + np.cos((np.pi*r)/(R)))
    else:
        h = 0.0

    lon_fun = 0.0
    lat_fun = 0.0
    f = 0.0

    return lon_fun,lat_fun, h, f
```

Listing A.11: visualization

```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import numpy as np
import math
from helperfuncs import *


def plot_global(uvwh, lonlat):

    lam = np.radians(lonlat[:,0])
    th = np.radians(lonlat[:,1])


    N = len(lam)
    xyz = getcartesian(lonlat)

    uvw = np.zeros((N,3))
    h = np.zeros(N)
    v_lon = np.zeros(N)
    v_lat = np.zeros(N)
```

```
#print("length of xyz: ", len(xyz))
x = xyz[:,0]
y = xyz[:,1]
z = xyz[:,2]

u = uvwh[:,0]
v = uvwh[:,1]
w = uvwh[:,2]
h = uvwh[:,3]

v_zonal, v_meri = cvec2gvec_sphere(uvwh,lam,th)

# Set up a 3D plot
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(x,y,z, c=h, cmap='jet')
fig.colorbar(scatter, ax=ax, label='Values')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Geopotential␣height' )
plt.savefig('height.png')
#plt.show()
plt.close()

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
scatter2 = ax.scatter(x,y,z, c=v_zonal, cmap='jet
')
fig.colorbar(scatter2, ax=ax, label='Values')

ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
ax.set_title('Zonal␣velocity' )
plt.savefig('zonal.png')
#plt.show()
plt.close()

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
scatter3 = ax.scatter(x,y,z, c=v_meri, cmap='jet'
)
```

```
    fig.colorbar(scatter3, ax=ax, label='Values')

    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.set_title('Meridional Velocity' )
    plt.savefig('meri.png')
    #plt.show()
    plt.close()
```

Listing A.12: WuWendland_functions

```
def wu0(r):
    k = 0
    l = k+2 #floor(d/2) + k + 1
    if (r <= 1):
        phi = (1-r**2)
    else:
        phi = 0
    return phi
def wendland0(r):
    k = 0
    l = k+2 #floor(d/2) + k + 1
    if (r <= 1):
        phi = (1-r)**l
    else:
        phi = 0
    return phi

def wendland1(r):
    k = 1
    l = k+2
    if (r <= 1):
        phi = (((1-r)**(l+1))*((l+1)*r +1))
    else:
        phi = 0
    return phi

def wendland2(r):
    k=2
    l=k+2
    if (r <= 1):
        phi = ((1-r)**(l+2))*(((l**2)+(4*l)+3)*(r**2)
         +(3*l + 6)*r + 3)
    else:
```

```
        phi = 0
    return phi

def wendland3(r):
    k=3
    l=k+2

    if (r <= 1):
        phi = ((1-r)**(l+3))*(((l**3) + 9*(l**2) +
        23*l + 15)*(r**3) + (6*(l**2) + 36*l + 45)*(r
        **2) + (15*l + 45)*r + 15 )
    else:
        phi = 0
    return phi

def wendland0_prime(r):

    if (r <= 1):
        phi_p = -2*(1 - r)
    else:
        phi_p = 0
    return phi_p


def wendland1_prime(r):

    if (r <= 1):
        phi_p = -16*(r**3) + 33*(r**2) - 18*r +1
    else:
        phi_p = 0
    return phi_p
```

Bibliography

[1] American Institute of Physics. Atmospheric general circulation modeling. 2008.

[2] Gianpaolo Balsamo. *Biosphere, Hydrosphere, and Cryosphere Models*. Reading, England, 2019.

[3] P. Bauer, A. Thorpe, and G. Brunet. The quiet revolution of numerical weather prediction. *Nature*, 525:47–55, 2015.

[4] Kaifeng Bi, Lingxi Xie, Hengheng Zhang, Xin Chen, Xiaotao Gu, and Qi Tian. Accurate medium-range global weather forecasting with 3d neural networks.

[5] D. S. Broomhead and David Lowe. Radial basis functions, multi-variable functional interpolation and adaptive networks. Technical Report 4148, RSRE, 1988.

[6] Martin Dietrich Buhmann. *Radial Basis Functions: Theory and Implementations*. Cambridge University Press, 2003.

[7] SciPy Contributors. SciPy documentation - scipy.linalg.lu. https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.lu.html, 2023. Accessed on: 10 Jan 2024.

[8] Henry Darcy. Les fontaines publiques de la ville de dijon. *Dijon: V. Dalmont*, 1856.

[9] Willem Deconinck. example-unstructured. https://github.com/wdeconinck/atlas4py/blob/feature/pointcloud/examples/example-unstructured.py, 2023.

[10] Willem Deconinck, Peter Bauer, Michail Diamantakis, Mats Hamrud, Christian Kühnlein, Pedro Maciel, Gianmarco Mengaldo, Tiago Quintino, Baudouin Raoult, Piotr K. Smolarkiewicz, and Nils P. Wedi. Atlas : A library for numerical weather prediction and climate modelling. *Computer Physics Communications*, 220:188–204, 2017.

[11] Lakshmi Aparna Devulapalli. Thesis implementation. https://github.com/dlaparna7/MyThesis2023, 2023.

[12] Encyclopedia of the Environment. Weather forecasting models.

[13] European Centre for Medium-Range Weather Forecasts (ECMWF). ECMWF Atlas Documentation. Accessed on: 10 Jan 2024.

[14] European Centre for Medium-Range Weather Forecasts (ECMWF). Ecmwf atlas: Partitioner, 2023.

[15] Gregory E. Fasshauer. *Meshfree Approximation Methods with MATLAB*. World Scientific Publishing Co. Pte. Ltd., Singapore, 2007.

[16] Natasha Flyer and Grady Wright. A radial basis function method for the shallow water equations on a sphere. 2009.

[17] Natasha Flyer, Grady B. Wright, and Bengt Fornberg. Radial basis function-generated finite differences: A mesh-free method for computational geosciences. *Journal Name*, 2010.

[18] R. A. Gingold and J. J. Monaghan. Smoothed particle hydrodynamics: Theory and application to non-spherical stars. *Monthly Notices of the Royal Astronomical Society*, 1977.

[19] What is Climate Change?

[20] E. J. Kansa. Multiquadrics—a scattered data approximation scheme with applications to computational fluid-dynamics—ii solutions to parabolic, hyperbolic and elliptic partial differential equations. *Computers & Mathematics with Applications*, 1990.

[21] Peter Lynch. The origins of computer weather prediction and climate modeling. *Journal of Computational Physics*, 2008.

[22] Tiruvenkadam Santhanam and A. C. Subhajini. An efficient weather forecasting system using radial basis function neural network.

[23] William Barton Sawyer. *Efficient Numerical Methods for the Shallow Water Equations on the Sphere*. Doctoral dissertation, Swiss Federal Institute of

Technology Zurich, Zurich, 2006. Submitted for the degree of Doctor of Sciences.

[24] Severe Weather Europe. Winter forecast 2023-2024 season: Early look at polar vortex, el niño, united states, canada, europe.

[25] M. Tillenius, E. Larsson, E. Lehto, and N. Flyer. A scalable rbf–fd method for atmospheric flow. *Journal of Computational Physics*, 298:406–422, 2015.

[26] Venkataramana Veeramsetty, Prabhu Kiran, Munjampally Sushma, and Surender Reddy Salkuti. Weather forecasting using radial basis function neural network in warangal, india.

[27] C. B. Vreugdenhil. *Numerical Methods for Shallow-Water Flow*, volume 13 of *Water Science and Technology Library*. Springer, 1986.

[28] Warren Washington, Lawrence Buja, and Anthony Craig. The computational future for climate and earth system models: On the path to petaflop and beyond. *Philosophical transactions. Series A, Mathematical, physical, and engineering sciences*, 367:833–46, 12 2008.

[29] Holger Wendland. Piecewise polynomial, positive definite and compactly supported radial functions of minimal degree. *Advances in Computational Mathematics*, 1995.

[30] David L. Williamson, John B. Drake, James J. Hack, Rodiger Jakob, and Paul N. Swarztrauber. A standard test set for numerical approximations to the shallow water equations in spherical geometry. *Journal of Computational Physics*, 1992.

[31] Zongmin Wu. Compactly supported positive definite radial functions. *Advances in Computational Mathematics*, 4(1):283–292, 12 1995.

[32] Sheng-Xin Zhu. Compactly supported radial basis functions: How and why?

The signed declaration of originality is a component of every semester paper, Bachelor's thesis, Master's thesis and any other degree paper undertaken during the course of studies, including the respective electronic versions.

Lecturers may also require a declaration of originality for other written papers compiled for their courses.

I hereby confirm that I am the sole author of the written work here enclosed and that I have compiled it in my own words. Parts excepted are corrections of form and content by the supervisor.

Title of work (in block letters):

Parallel implementation of mesh-free operators for 2D and 3D PDEs on a sphere for atmospheric dynamics

Authored by (in block letters):

For papers written by groups the names of all authors are required.

Name(s):	**First name(s):**
Devulapalli Lakshmi Aparna	Lakshmi Aparna

With my signature I confirm that

- I have documented all methods, data and processes truthfully.
- I have not manipulated any data.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for plagiarism.

Place, date	**Signature(s)**
Lugano, 10 January, 2024	Laparna.

For papers written by groups the names of all authors are required. Their signatures collectively guarantee the entire content of the written paper.